

UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

Department of Computing & Technology

Investigate, Fix or Escalate: A Rule-Constrained Reinforcement Learning Agent for IT Incident Triage under Ambiguous Evidence

By

OPOL CHARIS
Reg No: *S24B23/003*, Student No: *B29654*
E-mail: *charisopol@gmail.com*, Phone No: *0765821211*

A proposal submitted to the Faculty of Engineering Design & Technology for the study leading to the Degree in partial fulfilment of the requirements for the award of **Bachelor of Science in Computer Science** of Uganda Christian University

SPECIALIZATION TRACK: *Artificial Intelligence and Robotics*

24th October 2026

Declaration

I, *OPOL CHARIS*, declare to the best of my knowledge that the work in this proposal is original and has not been submitted before to any university/institution. It is the result of my effort.

Signature: _____

Date: _____

Ethical Generative AI Usage Declaration

A. **Declaring the use of AI in preparing this proposal**

I, *OPOL CHARIS*, declare that generative AI was used in preparing this proposal only in the ways disclosed in Appendix C: to review my arguments and suggest improvements to the clarity and structure of my work; to assist with the systematic literature review, including refining search strategies, reviewing literature and strengthening the discussion of research gaps; and to write and format LaTeX code for tables, generate some figures and improve the document layout. I evaluated every suggestion and output, and retained or modified it based on my own research and understanding.

Signature: _____ Date: _____

B. **Acknowledging the use of AI as a study buddy**

Appendix C gives the complete declaration of AI usage, including the tool used and the purposes for which it was used.

Approval

This proposal has been submitted for examination with approval of the following advisors/supervisor.

Name: _____
Department of Computing & Technology
Faculty of Engineering Design & Technology
Uganda Christian University

Signature: _____

Date: _____

Abstract

When an IT incident is reported, the staff who receive it must decide what to do before they know its cause: investigate further, apply a likely fix, or pass the incident to someone better placed to resolve it. The first evidence is often incomplete, unreliable or consistent with more than one cause, yet most automated triage tools treat the decision as a single prediction from the incident report. They cannot ask for more evidence, cannot say that they are unsure, and do not check their suggestions against an organisation's operational rules.

This research proposes to design and evaluate a rule-constrained reinforcement learning agent that, for each incident, learns whether to request a diagnostic check, apply a fix that the operational rules permit, or escalate the incident to a human expert. An independent rule-based safety mechanism checks every proposal, returns a blocked proposal to the agent once, and escalates the incident if the second proposal is also blocked. The agent will be trained and evaluated in a simulation built from incident scenarios elicited from the Directorate of University ICT Services at Uganda Christian University, in which missing, unreliable and overlapping evidence can each be controlled, and compared with six baseline policies on decision quality, investigation efficiency, escalation behaviour and constraint compliance. A prototype places the trained agent between an incident ticket and the action taken on it, carrying out its fixes only as draft changes to a fictional repository that a person reviews. In a systematic review of 34 studies, conducted for this research, no reviewed agent combined these three decisions for IT service incidents under explicit rules.

Dedication

I dedicate this research proposal to my father, Rev. Canon Dr Samuel Opol.

Contents

Declaration i

Ethical Generative AI Usage Declaration ii

Approval iii

Abstract iv

Dedication v

Acknowledgments ix

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Objectives . . . 2
  - 1.3.1 Scenario Model and Simulated Environment: . . . 2
  - 1.3.2 Rule-Constrained Agent: . . . 2
  - 1.3.3 Evaluation against Baselines and under Ambiguity: . . . 3
- 1.4 Research Questions . . . 3
- 1.5 Research Hypotheses . . . 3
- 1.6 Justification . . . 4

2 Literature Review 5
- 2.1 Literature Review Plan . . . 5
- 2.2 Review of Approaches to Incident Triage . . . 6
  - 2.2.1 Foundations of Safe Reinforcement Learning (2015–2018) . . . 6
  - 2.2.2 Information Gathering and Deferral (2018–2023) . . . 6
  - 2.2.3 Triage as Prediction in AIOps (2020–2025) . . . 7
  - 2.2.4 Reinforcement Learning for Incident Handling (2023–2026) . . . 7
  - 2.2.5 Recent Combinations (2025–2026) . . . 8
  - 2.2.6 Evaluation and Robustness . . . 9
- 2.3 Research Gaps . . . 10

3 Methodology 11
- 3.1 Overview . . . 11
- 3.2 Data Collection . . . 12
- 3.3 Problem Definition . . . 12
  - 3.3.1 Triage as a Partially Observable Decision Problem: . . . 12
  - 3.3.2 Belief over the Hidden Cause: . . . 13

3.3.3 Operational Rules and the Objective: . . . 14
3.4 The Simulated Triage Environment . . . 14
3.5 The Rule-Based Safety Mechanism . . . 16
3.6 The Reinforcement Learning Agents . . . 17
3.6.1 Deep Q-Network: . . . 17
3.6.2 Proximal Policy Optimisation: . . . 18
3.6.3 Training: . . . 19
3.7 Baseline Decision Policies . . . 20
3.8 Prototype: the Agent as a Decision Layer . . . 21
3.9 Evaluation Strategy . . . 21
3.9.1 Decision Quality: . . . 23
3.9.2 Investigation Efficiency: . . . 23
3.9.3 Escalation Behaviour: . . . 23
3.9.4 Constraint Compliance: . . . 24
3.9.5 Robustness: . . . 24
3.9.6 Reporting: . . . 25
3.9.7 Decision Criteria: . . . 25
3.9.8 Experiments: . . . 25
3.10 Limitations . . . 26

4 Expected Results **27**
4.1 Decision Quality . . . 27
4.2 Rule Compliance . . . 27
4.3 Robustness to Ambiguity . . . 27
4.4 Pilot Results . . . 28
4.5 Prototype . . . 28

REFERENCES **31**

APPENDICES **31**

A Budget **32**

B Research Project timelines **33**

C Declaration of AI use **35**

D List of 10 Major Publications Reviewed **36**

E Scopus Search Strings used during the literature review **38**
E.1 Scopus and ACM Digital Library (all fields) . . . 38
E.2 IEEE Xplore (open access only) . . . 38

F Research alignment with specialization, SDG, NDPIV & Vision 2040 **39**
F.1 Alignment with programme specialization . . . 39
F.2 Alignment with Sustainable Development Goals . . . 39
F.3 Alignment with NDP IV . . . 39
F.4 Alignment with the Uganda Vision 2040 . . . 39

G Algorithms, Code of interest **40**
G.1 Simulating an Incident . . . 40
G.2 Checking Actions against the Rules . . . 41

G.3 Training the Agent 42
G.4 Confidence-Threshold Baseline 42
G.5 Evaluating the Policies 43
G.6 Code Repositories 43

H Other useful figures **44**

I Research poster layout **45**

Acknowledgments

I want to thank God Almighty for strengthening me to work on this proposal and for the direction this research has taken. His grace started the good work, and grace will finish it.

I also thank my parents, Rev. Dr Samuel Opol and Margaret Opol, for the support they have given me, rallying me on when things got tough.

I thank Rebecca Kangabe, the Director of University ICT Services, Kenneth Musasizi, the Technical Lead at University ICT Services, and Simon Lubambo, the Senior Developer at University ICT Services, for their support of this research.

Finally, I thank Emmanuel Isabirye for redirecting my research.

Chapter 1

Introduction

1.1 Background

Organisations depend on IT services that are expected to remain available, and incident management is the process that restores those services when they fail. Chen et al. describe it as detecting a service problem, creating an incident, determining its cause, restoring the service, and capturing knowledge to prevent a recurrence [1]. Incident triage is the stage at which a reported incident is assessed and directed towards resolution [1, 2]. It is a set of decisions made before the cause of the incident is known.

These decisions are difficult because the evidence available when they are made is often incomplete or conflicting. In a two-year study of incident management at Microsoft, Chen et al. found that distinct problems can produce similar symptoms, so that the symptoms alone may not identify the responsible team, and incidents are reassigned several times as a result [1]. Remil et al. note that data in real IT environments is commonly noisy and incomplete [2]. At the Directorate of University ICT Services (UIS) at Uganda Christian University, incidents reach developers through user tickets that are vetted and assigned, and application errors are diagnosed from system logs and a monitoring dashboard. An initial interview with a senior developer of the Directorate in October 2026 described this process and identified the application programming interface logs as the most informative source of evidence.

Reinforcement learning (RL) offers a way to treat triage as a sequence of decisions. In RL, an agent learns a policy, which is a rule for choosing actions, by acting in an environment and receiving rewards that reflect the consequences of its actions [3]. When the agent cannot observe the true state, it can choose actions for the information they yield, and weigh the cost of that information against its value [4]. Under this framing, an agent faced with an ambiguous incident can gather more evidence, apply a fix, or hand the incident to a person.

1.2 Problem Statement

IT support teams must decide how to handle an incident before they know its cause, because its symptoms are often not enough to identify the problem: distinct problems can produce similar symptoms [1]. A member of IT staff facing such an incident can investigate further, apply a likely remedy, or pass the incident to someone better placed to resolve it, and each choice has a cost. Further investigation takes time, since engineers often need a long time to identify an incident's root cause [1]. A wrong remedy can reduce availability instead of restoring it [5], and an incident sent to the wrong team is reassigned, which prolongs the downtime [1]. Escalation

draws on the limited attention of the people who receive it, and when they are overloaded, responses are delayed and significant incidents are missed [6, 7].

Existing automated approaches address parts of this decision but not the choice among its options. Triage models commonly produce one prediction per incident report and are evaluated with classification metrics [1, 2], so they cannot ask for more evidence and cannot say that they are unsure. Reinforcement learning has been applied to incident response and investigation, mostly in cybersecurity [8–10], and a few studies of security alert triage let an agent gather evidence and then escalate or defer an alert to an analyst [6, 11]. Their agents close or escalate alerts, however, and do not apply a fix that changes the affected system. A systematic review of 34 studies, conducted for this research, found escalation to a person in only seven of them, and only three that report what happens when a rule mechanism blocks a decision. A reward penalty alone does not ensure that an agent respects an organisation's operational rules, because penalising an action does not guarantee that the agent never selects it [12].

As a result, the reviewed literature contains no evaluated agent for IT service incidents that learns when to investigate, when to apply a permitted fix and when to escalate, with its proposals checked against explicit operational rules, and no evidence on how such an agent behaves as the evidence becomes more ambiguous. This research seeks to address that gap. It does not claim that constraining a learned incident-handling agent is new; its contribution lies in the decision it studies, the setting, and the way the result is evaluated.

1.3 Research Objectives

The general objective of this study is to design and evaluate a rule-constrained reinforcement learning agent for ambiguity-aware IT incident triage that learns when to gather additional diagnostic information, apply permitted fixes, or escalate incidents to a human expert. It is guided by the following specific objectives.

1.3.1 Scenario Model and Simulated Environment:

To build a simulated triage environment from a scenario model of about 15 causes, 20 diagnostic checks and 10 operational rules elicited from the Directorate and reviewed by its staff, in which an agent can request diagnostic information, apply permitted fixes or escalate incidents, and in which missing, unreliable and overlapping evidence can each be set independently. This objective addresses how ambiguous triage scenarios can be represented in a controlled environment.

1.3.2 Rule-Constrained Agent:

To train a deep Q-network agent that chooses among requesting checks, applying fixes and escalating, under an independent rule-based safety mechanism that returns a blocked proposal to the agent once and escalates the incident if the second proposal is also blocked, so that no prohibited action is executed, and to repeat the training with proximal policy optimisation as a robustness check on the choice of learning algorithm. This objective addresses how reinforcement learning can support the three decisions and how a rule mechanism affects the agent's decisions.

1.3.3 Evaluation against Baselines and under Ambiguity:

To evaluate the agent against six baseline decision policies on decision quality, investigation efficiency, escalation behaviour and constraint compliance, and to measure how its accuracy and its mix of investigating, fixing and escalating change across controlled levels of missing, unreliable and overlapping evidence. This objective addresses how the approach compares with simpler policies and how robust it is as the evidence becomes more ambiguous.

1.4 Research Questions

This study will answer the following research questions:

1. How can ambiguous IT incident scenarios, built from the causes, diagnostic checks and operational rules of the Directorate, be represented in a simulated environment in which missing, unreliable and overlapping evidence can each be controlled?
2. How effectively can a reinforcement learning agent learn when to request diagnostic information, when to apply a permitted fix, and when to escalate an incident to a human expert, and do the findings hold when the deep Q-network is replaced by proximal policy optimisation?
3. How does a rule-based safety mechanism, which returns a blocked proposal to the agent once and escalates the incident after a second block, affect the agent's compliant success, its rate of blocked proposals and its decisions, compared with an agent without the mechanism and an agent that learns the rules only from a reward penalty?
4. How does the agent compare with the selected baseline decision policies, and how do its accuracy and its mix of investigating, fixing and escalating change as the evidence becomes more ambiguous?

1.5 Research Hypotheses

This study will test the following hypotheses. The criteria by which each is judged supported or not supported are stated in Section 3.9.7.

H1: At the base level of ambiguity, the agent will achieve a higher resolution accuracy than the initial-symptom, scripted-checklist and always-escalate baseline policies, with a mean investigation cost no higher than that of the scripted checklist.

H2: The agent with the safety mechanism will achieve a higher compliant success rate, the share of incidents resolved correctly without any prohibited action taking effect, than the unconstrained and penalty-only agents.

H3: The agent's response to ambiguity will depend on its source:

- **H3a:** as more evidence goes missing, it will escalate more incidents, without requesting more checks, since unavailable checks cannot be requested;
- **H3b:** as evidence becomes less reliable, it will request more checks before deciding and apply fixes on fewer incidents;
- **H3c:** as the evidence for different causes overlaps more, it will request more checks and escalate more incidents.

That the safety mechanism prevents every prohibited action is not a hypothesis, because it holds by construction (Equation (3.14)). It is checked as a verification of the implementation: the executed-violation rate must be zero on every evaluation incident in which the mechanism is active.

1.6 Justification

This research is justified by a decision that IT support teams face routinely and that existing automated approaches represent only in part. The literature shows that incident evidence is often insufficient to identify the cause [1], and that automation in AIOps mainly stops at triage or the recommendation of solutions, with automatic execution limited and unverified [13]. The move from recommending to acting is therefore the least settled step, and it is the step this study takes: its agent applies the fix it chooses, under operational rules, instead of handing a suggestion to a person. Reinforcement learning and explicit rules compensate for each other's weaknesses: the learner can judge when further investigation is worth its cost, which fixed rules handle poorly, and the rules give an assurance about prohibited actions that a learned policy cannot [14]. Keeping the rules in a mechanism separate from the learner also makes them readable and changeable by IT staff.

The study is feasible and low-risk because it is carried out entirely in simulation. Simulation allows the level of ambiguity to be controlled and experiments to be repeated, without acting on any live system of the university. Working with the Directorate grounds the simulated scenarios and rules in the practice of an operating IT support unit. The study also supports the use of artificial intelligence in Uganda's public and institutional IT services, which Appendix F relates to the programme specialisation and to national and global development goals.

Chapter 2

Literature Review

2.1 Literature Review Plan

The review was planned around the decision this research studies: what an agent should do with an IT incident whose cause is uncertain. It drew on two sources. The first was a core set of publications on AI for IT operations (AIOps), safe reinforcement learning and learning to defer, collected while the problem was being defined. The second was a systematic review, reported following the PRISMA 2020 guideline [15], which searched Scopus, the ACM Digital Library and IEEE Xplore, with the Scopus and ACM searches limited to publications from 2015 to 2026, and included 34 studies that apply reinforcement learning or sequential decision-making to IT or security incident handling. The search strings are given in Appendix E and the flow of records in Appendix H.

The literature is discussed chronologically, in the order in which the strands developed: the foundations of safe reinforcement learning, then information gathering and deferral, then the prediction-based approaches of AIOps, then reinforcement learning for incident handling, and finally the recent studies that begin to combine these strands. For each approach the review states what it does, what evidence supports it and where it falls short, and then judges it against five requirements that an agent for IT incident triage at a university IT directorate must meet:

R1. Sequential decisions: the agent decides step by step, and each step can change what it knows.

R2. Costly investigation: the agent can request a diagnostic check and weigh what the check costs against what it may reveal.

R3. Remedy or hand-over: the agent can apply a fix that changes the affected system, or hand the incident to a human expert.

R4. Operational rules: every proposed action is checked against the organisation's rules, and a prohibited action is never carried out.

R5. Ambiguous evidence: the agent stays reliable as the evidence becomes missing, unreliable or consistent with several causes.

2.2 Review of Approaches to Incident Triage

2.2.1 Foundations of Safe Reinforcement Learning (2015–2018)

These methods keep a learning agent from taking prohibited actions, and they are the source of the rule mechanism in this study (R4).

- **Constrained optimisation (2015–2017):** García and Fernández divided safe reinforcement learning into methods that change the optimisation criterion and methods that change exploration with external knowledge [16]. Constrained policy optimisation, from the first group, bounds the expected cumulative cost of a policy and guarantees near-constraint satisfaction at every training iteration [17]. Its strength is that safety is built into training rather than added afterwards. Its weakness is that the guarantee holds on average: an expected-cost bound does not enforce a rule that must hold at every step [18], and a reward penalty discourages an action without preventing it [12]. The rules of an IT directorate are not averages; a rule such as "do not restart a production service without approval" must hold on every incident, so this approach cannot meet R4 on its own.
- **Shielding (2018 onwards):** Alshiekh et al. introduced a shield, a separate component that checks each action the agent proposes and blocks or replaces it when it would violate a specification, so that safety holds during both learning and deployment while the agent is restricted as little as possible [14]. For this study the main strength is the separation: the rules sit outside the learner, where IT staff can read and change them. Three weaknesses remain. The shield is computed from an abstraction of the environment's dynamics [14], and shields that need a full model of the environment cannot be applied where no such model exists [19]. A shield that silently replaces actions can hinder learning, because the agent does not know it was overridden [19]. And shields have been demonstrated mainly in control tasks and games [12, 14], not on the rules of IT operations. The rules of an IT directorate are conditions on the proposed action and the evidence gathered so far, so they can be checked without a model of how incidents evolve, but what happens to a blocked action has to be designed for this setting rather than borrowed.

Constrained optimisation shows that a penalty alone cannot meet R4, and shielding shows how R4 can be met. Neither addresses investigation, hand-over or ambiguous evidence.

2.2.2 Information Gathering and Deferral (2018–2023)

These methods let a model delay or hand over its decision, the two options that prediction-only triage lacks (R2 and R3).

- **Learning to defer (2018):** Madras et al. trained a model that can pass a decision to an external decision-maker and that accounts for the biases of that decision-maker; deferring made the combined system both more accurate and fairer, even when the human was inconsistent [20]. Joshi et al. later extended deferral to sequential settings [21], and selective prediction gives a model the related option to abstain when its confidence is low [22]. The strength of this line is that handing over is learned rather than set by a fixed threshold, and it is judged by how well the human and the model perform together. Its weakness for incident triage is that the model can only decide or hand over; it cannot ask for more evidence first. Madras et al. evaluated single classification decisions on recidivism and health datasets [20], where no further evidence can be collected. Applied to IT incidents, deferral alone would hand over incidents that one more check could have resolved.

- **Information gathering (2019):** Janisch et al. trained an agent to acquire costly features one at a time before it classifies [23], building on the principle that, under partial observability, an action may be chosen for the information it yields [4]. Its strength is that investigation is treated as a decision with a cost (R2), which is close to how IT staff work through diagnostic checks. Its weakness is the mirror image of deferral: the agent can look further or decide, but it cannot hand over, so when the evidence stays ambiguous it must still commit to an answer.

The two lines complement each other. Each supplies one of the options this study needs, neither supplies both, and neither checks its decisions against operational rules.

2.2.3 Triage as Prediction in AIOps (2020–2025)

These methods treat triage as the prediction of a label from the incident report.

- **Incident management at scale (2020):** Chen et al. conducted an empirical study of more than two years of incident management at Microsoft [1]. Its strength is its evidence: drawn from real operations, it shows that distinct problems can produce similar symptoms, so that incidents are reassigned several times, which is the ambiguity this study simulates. The remedy it describes, however, is a recurrent model that predicts the responsible team from the incident text [1]. Such a model makes one prediction per incident; when the text is ambiguous it still names a team, and the reassignments the study documents are the cost of naming the wrong one.
- **AIOps surveys (2024–2025):** Remil et al. catalogued the methods for classifying, prioritising and assigning incidents, and criticised the reliance on classification metrics that ignore the cost of a wrong action [2]. Zhang et al. observed that language-model assistants extended this work to diagnosis and mitigation suggestions, but that automation mainly stops at triage or recommendation [13]. The surveys are valuable as maps of the field and for the measurement problem they identify. The methods they survey, though, return an answer for every report and are scored on how often that answer is right. They cannot state that more evidence is needed or that a person should decide (R2, R3), and accuracy says nothing about whether an applied fix was permitted (R4).

This group supplies the problem and the evidence for it, but not a model of the decision. It corresponds to the single-prediction baseline against which this study's agent is compared.

2.2.4 Reinforcement Learning for Incident Handling (2023–2026)

These studies treat incident handling as a sequence of decisions (R1).

- **Root cause analysis (2023):** Wang et al. used hierarchical reinforcement learning to decide which questions to put to engineers when building a service dependency graph for Microsoft's microservice systems, reducing the number of queries needed from $O(N^2)$ to $O(1)$ [24]. Ding et al.'s TraceDiag learned to prune the dependency graph before analysis and was deployed in production at Microsoft [25]. Both treat investigation as a decision with a cost (R2), and both were built on real production systems, which few studies in this review achieve. Their goal, however, is to locate a root cause, not to decide what to do about it: neither applies a remedy, checks one against rules, or decides whether a person should take over. Both also rely on the dependency graphs and traces of large microservice systems, whereas the Directorate diagnoses errors from API logs and a monitoring dashboard.

- **Security incident response (2025–2026):** Dunsin et al. trained an agent to conduct a malware investigation step by step, aiming to shorten post-incident forensic investigations, and evaluated it in simulated scenarios [8]. Hammar and Li planned incident responses while holding a belief over the model of the incident itself, relaxing the usual assumption that the system model is known, and bounded the errors this introduces [9]. Hammar et al. adapted response policies to changes in the system without repeating the offline optimisation [10]. The Hammar studies are the strongest in this group on R5: on the CAGE-2 benchmark, and for the 2026 study also on a testbed, they outperformed earlier methods in robustness to an inaccurate model [9, 10]. Their weaknesses for this study are twofold. Their agents choose among responses or investigative steps, but none can hand the incident to a person (R3), and none checks its actions against an organisation's rules (R4). Their environments also model cyberattacks, in which an adversary drives the incident, whereas the IT service incidents in this study arise from faults and misconfigurations.

This group shows that reinforcement learning can handle investigation and response as sequential decisions. In every study, however, the decision ends with the agent: none passes an incident to a person or respects operational rules.

2.2.5 Recent Combinations (2025–2026)

The most recent studies begin to combine the strands above, mostly in security operations.

- **Gathering evidence and escalating in alert triage:** Okafor trained offline agents that dismiss, investigate or contain security alerts, with escalation to analysts fixed by design [26]. Its strength is methodological: it avoids unsafe online exploration, removes temporal leakage from the evaluation, and shows that F1 and ROC-AUC can hide degenerate policies [26]. Its weakness is its data: public intrusion datasets hold flow records rather than decisions, so its training episodes are reconstructed from flows [26], and the escalation it reports was not learned. Sahin and Mert trained an agent that queries logs and threat intelligence before it escalates or closes an alert, with a separate conformal layer that defers uncertain alerts to an analyst while controlling the rate of missed threats [11]. This gives hand-over a statistical guarantee rather than a tuned threshold, the strongest treatment of R3 in the review, but the deferral sits outside the learned policy: the agent itself is not taught when handing over is worth its cost. Kovtun et al. learned a policy that chooses among shallow triage, deep analysis and deferral under analyst-capacity constraints, and theirs is the one study that reports decision quality, cost, escalation and compliance together [6]. Its evaluation, like Okafor's, rests on public intrusion datasets, CIC-IDS2017 and UNSW-NB15 [6]. Across the three, the actions close, contain or escalate an alert; none applies a repair to a failing service, which is the action an IT directorate most needs to check against its rules.
- **Rule mechanisms in incident response:** Ravi et al.'s SAFE-HGX added a safety layer to a hierarchical multi-agent learner for intrusion response in industrial control systems, replacing blocked actions, and in simulated attack scenarios it learned faster and contained attacks more accurately than DQN and PPO baselines [27]. The cost was high: about 9 to 11 times as many false-positive actions as the PPO baseline, and 74 to 77% higher operating cost [27]. An IT directorate is unlikely to accept that trade-off, since an unnecessary action on a live service can itself disrupt operations [26]. Singh's deterministic guard for cloud remediation hands an action to a human when it fails [28]. In its ablation, removing the guard left the mean time to resolve almost unchanged (144 s against 142 s) but cut governance compliance from 99.9% to 72.3% [28], the clearest

evidence in the review that a hard rule check need not slow resolution. The learning in Singh's framework, however, is confined to a reinforcement learning meta-orchestrator that weights the recommendations of other agents [28], so the study does not show how a learning agent behaves when its own proposals are blocked. Alauthman et al. proposed a shielded DQN agent with operator override for incident response in industrial control systems, but reported an evaluation plan and indicative findings rather than measured results [5]; it shows that the design is plausible, not that it works. Of the 34 studies in the systematic review, only three say what happens to a blocked decision, and none lets the agent propose again.

These two groups have reached the design questions of this study from opposite sides. The alert-triage studies hand cases over but do not check remedies against rules; the rule-mechanism studies check actions against rules but do not learn when to hand over. Their results also disagree on what a rule mechanism costs: Ravi et al. found many more false actions, while Singh found no loss of speed.

2.2.6 Evaluation and Robustness

These studies show how such agents have been evaluated, and where the evaluations fall short.

- **Measures:** In the systematic review, 31 of 34 studies measure decision quality, but only five measure escalation behaviour and four constraint compliance; one study, Kovtun et al., reports all four kinds of measure, for SOC alert escalation [6]. This matters because each measure on its own can be satisfied by a poor policy: an agent that escalates every incident never breaks a rule and never applies a wrong fix, but it is of no use to the people it escalates to. A single metric can also hide a degenerate policy: Okafor found that F1 and ROC-AUC concealed agents that had collapsed into trivial detectors, which the Matthews correlation coefficient exposed [26].
- **Robustness:** Fifteen of the reviewed studies test their agents under changed conditions, and the methods that keep a belief over the hidden state degrade least [9, 10]. Sahin and Mert show that deferral to analysts rises as the evidence degrades [11]. These tests vary the conditions as a whole; no reviewed study reports how the mix of investigating, acting and escalating changes as ambiguity increases, which is the behaviour that shows whether an agent recognises when it does not know enough.

Table 2.1 compares the studies closest to this research against the capabilities it requires.

Table 2.1: Comparison of the studies closest to this research. ✓ = addressed; ○ = partly addressed; – = not addressed.

Study	**Setting**	**Seq.**	**Info.**	**Rule**	**Esc.**	**Amb.**
Chen et al. [1]	IT incident triage	–	–	–	–	–
Wang et al. [24]	Root cause analysis	✓	✓	–	○	–
Dunsin et al. [8]	Malware investigation	✓	✓	–	–	–
Hammar and Li [9]	Security incident response	✓	–	–	–	✓
Alauthman et al. [5]	Industrial control incident response	✓	–	✓	○	–
Ravi et al. [27]	Industrial control incident response	✓	–	✓	–	–
Okafor [26]	Security alert triage	✓	✓	○	○	–
Sahin and Mert [11]	Security alert triage	✓	✓	○	✓	✓
Kovtun et al. [6]	Security alert escalation	✓	○	○	✓	–
Singh [28]	Cloud operations remediation	○	–	✓	○	–
This study	IT incident triage	✓	✓	✓	✓	✓

Seq. = sequential policy; Info. = information gathering; Rule = rule check; Esc. = human escalation; Amb. = varied ambiguity.

2.3 Research Gaps

The approaches reviewed above each solve part of the triage decision, but no reviewed study combines them for IT service incidents. The most significant gap is in the decision itself: information-gathering studies let an agent choose between looking further and deciding, deferral studies let it choose between deciding and handing over, and the security alert-triage studies that join the two do not apply a fix that acts on the system. No reviewed study trains one policy to choose among investigating, applying a remedy and escalating for IT service incidents. The second gap concerns the rules: hard rule mechanisms have been measured in only two studies, their results point in opposite directions, and no reviewed study gives a blocked agent a second proposal or reports how often blocked proposals occur during training. The third gap concerns robustness: decision accuracy and the rate of deferral have been tested under degraded evidence, but not the mix of investigating, fixing and escalating as ambiguity rises. Underneath all three lies a gap in representation, since the available environments model cyberattacks and microservice faults, and none lets the ambiguity of IT service incidents be varied.

These gaps are stated for the literature reviewed. The systematic review searched three databases, limited its IEEE Xplore search to open-access records, did not carry out citation searching, and could not retrieve seven reports. The claim is that the reviewed literature leaves these questions open, not that they have never been examined.

Chapter 3

Methodology

3.1 Overview

To study how an agent should handle IT incidents whose cause is uncertain, this research adopts a design-and-evaluate approach [29]. It builds an artefact, a rule-constrained reinforcement learning agent together with the simulated environment in which it acts, and evaluates it in controlled experiments. The work proceeds in eight phases, shown in Figure 3.1: specifying the problem, eliciting scenarios with the Directorate, building and validating the environment, encoding the rules, implementing the baselines, training the agents, running the experiments, and analysing the results.

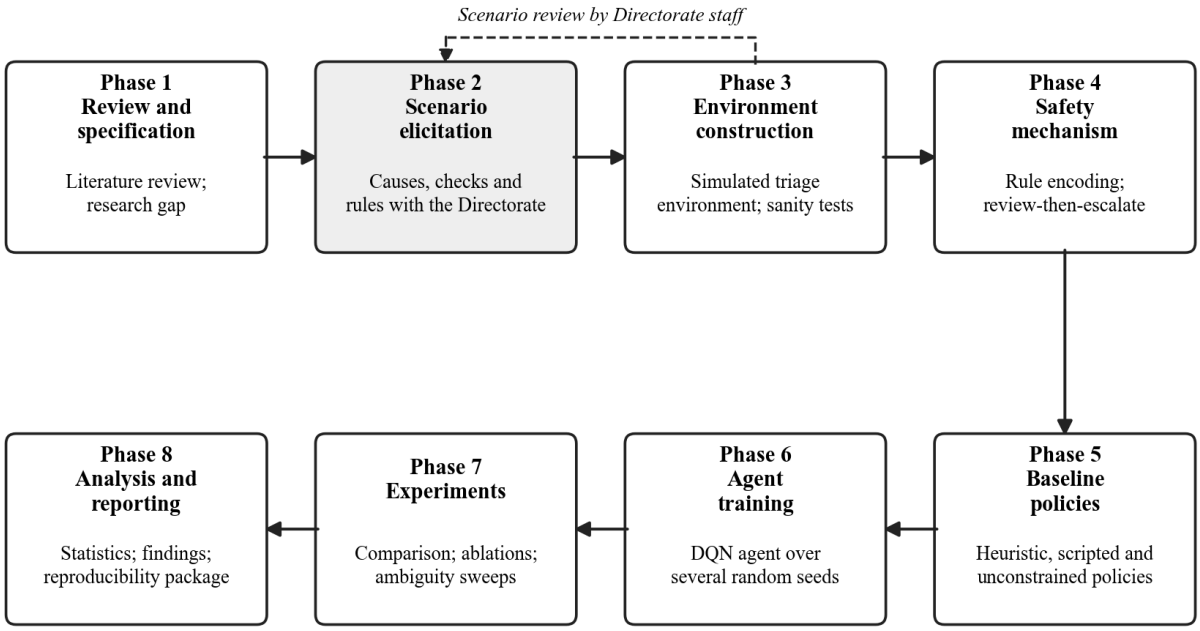


Figure 3.1: Research pipeline

The evaluation is conducted entirely in simulation, for two reasons. Realistic incident data is scarce and confidential [2], and simulation lets the level of ambiguity be set, held constant and

varied, which live incidents would not allow. The agent is never connected to any system of the university. The framework will be implemented in Python, with the Gymnasium interface for the environment [30] and the Stable-Baselines3 library for the learning algorithms [31]. No personal data, credentials or confidential details of the university's systems will enter the simulator, and staff participation will be voluntary and with consent.

3.2 Data Collection

The data for this research is a scenario model elicited from the Directorate of University ICT Services, not a dataset of incident records. The model will be elicited from Directorate staff using a structured workbook, and it will contain about 15 causes in four provisional categories (application and interface errors, authentication and access failures, database failures, and failures of infrastructure and external services), each with one correct fix; 20 diagnostic checks, such as an entry in the API logs, a value on the monitoring dashboard or the result of reproducing a failing request, each with an effort cost; a fixed list of about 10 symptoms, the signs reported when an incident is raised, such as a failed login or a slow page; and 10 operational rules.

Staff will give three sets of ratings on a five-point scale (never, rarely, sometimes, often, almost always) that is mapped to the probabilities 0.05, 0.25, 0.5, 0.75 and 0.95. For each pair of cause and check, they will rate how often the check is positive when that cause is present (300 ratings); for each pair of cause and symptom, how often the symptom is reported when that cause is present (150 ratings); and for each cause, how often it occurs, with the mapped values normalised to sum to one to give the prior over causes (15 ratings). Asking for verbal ratings keeps these 465 judgements manageable for practitioners. Any parameter that the elicitation does not supply will be set by the researcher and reported as such. After the environment is built, sample incidents will be shown to Directorate staff, who will judge whether they are plausible and whether the rules are stated correctly. The model will be revised and then frozen before any agent is trained, so that the scenarios cannot be adjusted in favour of the proposed agent.

3.3 Problem Definition

This section states the triage problem in general terms, independent of the numbers the Directorate will supply. Section 3.4 then gives the specific instance that is simulated.

3.3.1 Triage as a Partially Observable Decision Problem:

The triage of one incident is modelled as a finite-horizon partially observable Markov decision process (POMDP) [4],

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, R, \Omega, \mathcal{O}, \gamma, H \rangle, \tag{3.1}$$

whose elements are defined below.

Causes, checks and context. Let $\mathcal{C} = \{c_1, \ldots, c_K\}$ be the set of possible causes of an incident, $\mathcal{D} = \{d_1, \ldots, d_J\}$ the set of diagnostic checks, with check d_j having cost $k_j > 0$, and $\mathcal{X}$ the set of incident contexts, such as whether the affected system is in production. The result of check d_j is positive with a probability that depends on the cause:

$$\theta_{jm} = P(d_j = \text{positive} \mid c_m), \qquad j = 1, \ldots, J, \quad m = 1, \ldots, K. \tag{3.2}$$

An incident also arrives with symptoms. Each of the E symptoms is either present or absent, $\sigma_e \in \{0,1\}$, and is reported with a probability that depends on the cause:

$$\varphi_{em} = P(\sigma_e = 1 \mid c_m), \qquad e = 1,\dots,E, \quad m = 1,\dots,K. \tag{3.3}$$

The symptoms and the check results are assumed to be independent of one another given the cause.

States ($\mathcal{S}$). A state $s_t = (c, x, \boldsymbol{\sigma}, \mathbf{z}, \mathbf{q}_t)$ consists of the hidden cause $c \in \mathcal{C}$, the context $x \in \mathcal{X}$, the symptom vector $\boldsymbol{\sigma} \in \{0,1\}^E$, the vector $\mathbf{z}$ of check results, both drawn once at the start of the incident and unchanged afterwards, and the vector $\mathbf{q}_t \in \{0,1\}^J$ recording which checks have been requested by step t. The cause c is never observed by the agent.

Actions ($\mathcal{A}$). The agent can request a check, apply the fix for one cause, or escalate:

$$\mathcal{A} = \mathcal{A}_{\text{check}} \cup \mathcal{A}_{\text{fix}} \cup \{a_{\text{esc}}\}, \qquad \mathcal{A}_{\text{check}} = \{a_1^{\text{chk}},\dots,a_J^{\text{chk}}\}, \quad \mathcal{A}_{\text{fix}} = \{a_1^{\text{fix}},\dots,a_K^{\text{fix}}\}, \tag{3.4}$$

so that $|\mathcal{A}| = J + K + 1$.

Transitions ($\mathcal{T}$). The cause, context and check results are fixed for the whole episode. Requesting check d_j sets $q_{t+1,j} = 1$ and reveals its result. Applying a fix or escalating ends the episode, and so does reaching the step limit H, which is treated as an escalation.

Observations (Ω, $\mathcal{O}$). At step t the agent receives

$$o_t = (\mathbf{u}_t, \boldsymbol{\sigma}, x, t, \beta_t), \tag{3.5}$$

where $u_{t,j} \in \{\text{not requested}, \text{unavailable}, \text{negative}, \text{positive}\}$ is the state of check d_j, $\boldsymbol{\sigma}$ is the vector of symptoms reported at the start, and $\beta_t \in \{0,1\}$ is set when the agent's previous proposal was blocked by the safety mechanism.

Reward (R). The reward expresses the costs that Chapter 1 attributed to each choice:

$$R(s_t, a_t) = \begin{cases} -k_j & \text{if } a_t = a_j^{\text{chk}}, \\ +r_{\text{correct}} & \text{if } a_t = a_m^{\text{fix}} \text{ and } c_m = c, \\ -\lambda_{\text{wrong}} & \text{if } a_t = a_m^{\text{fix}} \text{ and } c_m \neq c, \\ -\lambda_{\text{esc}} & \text{if } a_t = a_{\text{esc}}, \\ -\lambda_{\text{block}} & \text{if } a_t \text{ is proposed and blocked by the safety mechanism,} \end{cases} \tag{3.6}$$

with

$$r_{\text{correct}} > -\lambda_{\text{block}} > -\lambda_{\text{esc}} > -\lambda_{\text{wrong}}, \tag{3.7}$$

so that resolving an incident correctly is better than escalating it, escalating it is better than applying the wrong fix, and a blocked proposal costs a little but less than an escalation. The block penalty $\lambda_{\text{block}} > 0$ gives the agent a reason to stop proposing prohibited actions; it is a learning signal, not the means of enforcement, which remains the safety mechanism. The simulated human expert is assumed to resolve every escalated incident. Future rewards are discounted by $\gamma \in (0,1]$.

3.3.2 Belief over the Hidden Cause:

Because the cause is hidden, the information available to the agent at step t is summarised by its belief, the posterior probability of each cause given what it has observed. Let $\mathcal{I}_t$ be the set of checks whose results have been observed by step t, and $z_j \in \{0,1\}$ the observed result of d_j.

All E symptoms are observed from the start. Since symptoms and results are fixed within an episode and independent of one another given the cause, Bayes' rule gives

$$b_t(c_m) = P(c_m \mid o_0, \ldots, o_t) \;\propto\; P(c_m) \prod_{e=1}^{E} \varphi_{em}^{\sigma_e} \, (1 - \varphi_{em})^{1-\sigma_e} \prod_{j \in \mathcal{I}_t} \theta_{jm}^{z_j} \, (1 - \theta_{jm})^{1-z_j}. \tag{3.8}$$

At $t = 0$, before any check, the belief rests on the prior and the symptoms alone. When the ambiguity controls of Section 3.4 are active, φ_{em} and θ_{jm} are replaced by their overlapped forms, θ_{jm} also by its noisy form, and unavailable checks are left out of $\mathcal{I}_t$. The belief depends only on the prior, the symptoms and the results observed so far, not on the order in which they were obtained. The observation o_t records exactly those results, so it is a sufficient statistic for the belief [4], and a policy over o_t loses nothing compared with a policy over the full history. The learning agent is not given θ_{jm} and does not compute Equation (3.8); it must learn the mapping from observations to actions. The confidence-threshold baseline of Section 3.7, which is given the model, computes the belief directly.

3.3.3 Operational Rules and the Objective:

The organisation's operational rules are a set $\{\rho_1, \ldots, \rho_{N_\rho}\}$ of conditions, each a function $\rho_q(a, x, \mathbf{u}_t) \in \{0, 1\}$ that equals 1 when it prohibits action a in context x given the evidence $\mathbf{u}_t$. The permitted actions at step t are

$$\mathcal{A}_{\text{safe}}(x, \mathbf{u}_t) = \{\, a \in \mathcal{A} : \rho_q(a, x, \mathbf{u}_t) = 0 \text{ for all } q = 1, \ldots, N_\rho \,\}. \tag{3.9}$$

Escalation is never prohibited, so $a_{\text{esc}} \in \mathcal{A}_{\text{safe}}$ always and a permitted action always exists.

A policy $\pi(a \mid o)$ gives the probability of proposing action a after observation o. The triage problem is to find the policy that maximises the expected discounted return while never executing a prohibited action:

$$\max_{\pi} \; \eta(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \right] \quad \text{subject to} \quad a_t^{\text{exec}} \in \mathcal{A}_{\text{safe}}(x, \mathbf{u}_t) \text{ for every step } t, \tag{3.10}$$

where $T \leq H$ is the length of the episode, r_t the reward at step t, and a_t^{exec} the action that is actually carried out. The constraint must hold on every step of every incident, not on average; this is why it is enforced by a separate mechanism (Section 3.5) rather than by a penalty in the reward [12, 18].

3.4 The Simulated Triage Environment

The environment is one instance of the problem in Section 3.3, built from the Directorate's scenario model and implemented with the Gymnasium interface [30]. One episode is the triage of one incident, generated and run as in Algorithms 1 and 2 (Appendix G). Table 3.1 lists the values that the environment gives to each element of the problem.

Table 3.1: How the simulated environment instantiates the problem

Element	**Value in the environment**
Causes K	About 15, in four categories, drawn from the elicited prior $P(c)$
Checks J	20, with costs k_j between 0.1 and 1.0
Rules N_ρ	10, elicited from the Directorate
Actions $\lvert\mathcal{A}\rvert$	$20 + 15 + 1 = 36$
Symptoms E	About 10, each present or absent
Likelihoods θ_{jm}, φ_{em}	Staff ratings mapped to 0.05, 0.25, 0.5, 0.75 or 0.95
Prior $P(c)$	Staff frequency ratings on the same scale, normalised to sum to one
Context x	Attributes such as whether the system is in production and the incident's priority
Reward	$r_{\text{correct}} = +10$, $\lambda_{\text{block}} = 0.5$, $\lambda_{\text{esc}} = 3$, $\lambda_{\text{wrong}} = 10$ as starting values, tuned in pilot runs within the ordering of Equation (3.7)
Evaluation	$N = 1{,}000$ incidents per evaluation set, $S = 10$ sets per ambiguity level (Section 3.9)
Step limit H and discount γ	Set in pilot runs and reported with the results

Generating an incident. At the start of an episode the environment draws a cause $c \sim P(c)$ and a context x, then draws each symptom with $P(\sigma_e = 1) = \varphi_{em}$ and the result of every check with $P(z_j = 1) = \theta_{jm}$ for the true cause c_m, each independently given the cause and each once. Requesting a check twice therefore returns the same result, and a check that has already been requested is removed from the agent's choices.

Encoding the observation. The observation o_t is presented to the agent as a fixed-length vector $\phi(o_t) \in \mathbb{R}^n$. Each check state $u_{t,j}$ is one-hot encoded over its four values, and each context attribute with more than two levels over its levels; the symptoms, binary attributes and the blocked flag β_t take the values 0 and 1, and the step count is divided by H. The length n is $4J + E$ plus the length of the context encoding plus two, and is fixed once the elicitation is complete.

Ambiguity controls. Each of the three sources of ambiguity identified in Chapter 2 has its own control:

- **Missing evidence:** with probability p_{miss}, a check is unavailable for the whole episode, and a request for it returns "unavailable".
- **Unreliable evidence:** with probability p_{noise}, the result of a check is reversed before the agent receives it, so the effective likelihood becomes

$$\tilde{\theta}_{jm} = (1 - p_{\text{noise}})\,\theta_{jm} + p_{\text{noise}}\,(1 - \theta_{jm}). \tag{3.11}$$

- **Overlapping evidence:** an overlap parameter ω moves the likelihoods of each cause towards the average of the causes in its category $g(m)$:

$$\theta'_{jm} = (1 - \omega)\,\theta_{jm} + \omega\,\bar{\theta}_{j,g(m)}, \qquad \bar{\theta}_{j,g(m)} = \frac{1}{|g(m)|} \sum_{m' \in g(m)} \theta_{jm'}, \tag{3.12}$$

  so that as ω increases, causes in the same category become harder to tell apart. The same blending is applied to the symptom likelihoods, giving φ'_{em}; missing and unreliable evidence apply to checks only.

A setting of the three controls, $\ell = (p_{\text{miss}}, p_{\text{noise}}, \omega)$, defines an ambiguity level. The planned ranges are $p_{\text{miss}} \in \{0, 0.2, 0.4, 0.6\}$, $p_{\text{noise}} \in \{0, 0.1, 0.2, 0.3\}$ and $\omega \in \{0, 0.25, 0.5, 0.75\}$. The base level ℓ_0 is one point of this grid, chosen in pilot runs so that the task is neither trivial nor impossible.

Reference labels. For evaluation only, each incident is labelled with whether it should be escalated:

$$y = \begin{cases} 1 & \text{if } a_c^{\text{fix}} \notin \mathcal{A}_{\text{safe}}(x, \mathbf{u}^{\text{all}}) \ \text{ or } \ \arg\max_m b^{\text{all}}(c_m) \neq c, \\ 0 & \text{otherwise,} \end{cases} \tag{3.13}$$

where a_c^{fix} is the fix for the true cause, and $\mathbf{u}^{\text{all}}$ and b^{all} are the evidence and the belief once every available check has been observed. In words, an incident should be escalated if the rules forbid the right fix, or if it stays ambiguous even with all the evidence. The labels are never shown to the agent. Before training, the simulator will be checked for leakage of the hidden cause into the observations, a defect that Sahin and Mert found in an earlier version of their own simulator [11]: no policy that sees only the observations can, on average, resolve more incidents correctly than the share labelled $y = 0$, so a policy that exceeds that ceiling by more than sampling error indicates leakage.

Fix with verification. In the formulation above a fix ends the episode, so a wrong fix is a final error. In practice a remedy is applied and then verified: the symptoms either clear or persist, and a person who has applied the wrong remedy goes on investigating. The study therefore also uses a variant of the environment in which a fix is followed by verification. Applying the remedy for cause c_m costs k_m^{fix}, the time it takes. If c_m is the true cause, the symptoms clear and the episode ends with $r_{\text{correct}} - k_m^{\text{fix}}$. Otherwise the agent pays $k_m^{\text{fix}} + h_m$, where h_m is the harm of the wrong remedy, observes that the fix failed, and continues; c_m is then ruled out and its fix is removed from the agent's choices. After n_{fail} failed fixes the incident passes to the human expert at the further cost λ_{esc}. The observation gains one indicator per cause that records a failed fix. The harms are anchored to the basic formulation: their mean is $\lambda_{\text{wrong}} - \lambda_{\text{esc}}$, so that on average a wrong fix followed by escalation costs what one wrong fix costs there, and they are spread across remedies according to how hard each is to undo, as stated in the reasons given for the operational rules. Verification is assumed to be reliable. In this variant the agent learns not only whether to fix but which remedy to try first, and the wrong-fix rate counts every wrong remedy applied, not only those that end an episode. The remedy times, the harm weights and n_{fail} are to be confirmed with the Directorate before the experiments.

3.5 The Rule-Based Safety Mechanism

The safety mechanism is a component separate from the agent that enforces the constraint in Equation (3.10). Each of the ten rules is a readable condition over the proposed action, the context of the incident and the evidence gathered so far; for example, a rule may forbid a fix on a production system, or forbid a fix until a named check has been observed. When the agent proposes an action $a_t^{(1)}$, a permitted action passes to the environment unchanged. A prohibited action is blocked and logged, earns the block penalty $-\lambda_{\text{block}}$ and uses up one step; the blocked flag β_t is set, the blocked action is masked, and the agent proposes once more, $a_t^{(2)}$. If the second proposal is also prohibited, the incident is escalated to the human expert and recorded as a forced escalation. The action carried out is therefore

$$a_t^{\text{exec}} = \begin{cases} a_t^{(1)} & \text{if } a_t^{(1)} \in \mathcal{A}_{\text{safe}}(x, \mathbf{u}_t), \\ a_t^{(2)} & \text{if } a_t^{(1)} \notin \mathcal{A}_{\text{safe}}(x, \mathbf{u}_t) \text{ and } a_t^{(2)} \in \mathcal{A}_{\text{safe}}(x, \mathbf{u}_t), \\ a_{\text{esc}} & \text{otherwise (forced escalation),} \end{cases} \tag{3.14}$$

which always lies in $\mathcal{A}_{\text{safe}}$, because escalation is never prohibited. Telling the agent about the block, rather than silently replacing its action, responds to the finding that a shield which silently overrides actions can hinder learning [19]. No study in the systematic review gives a blocked agent a second proposal. The procedure is given as Algorithm 3. Figure 3.2 shows how the agent, the safety mechanism and the simulated environment work together.

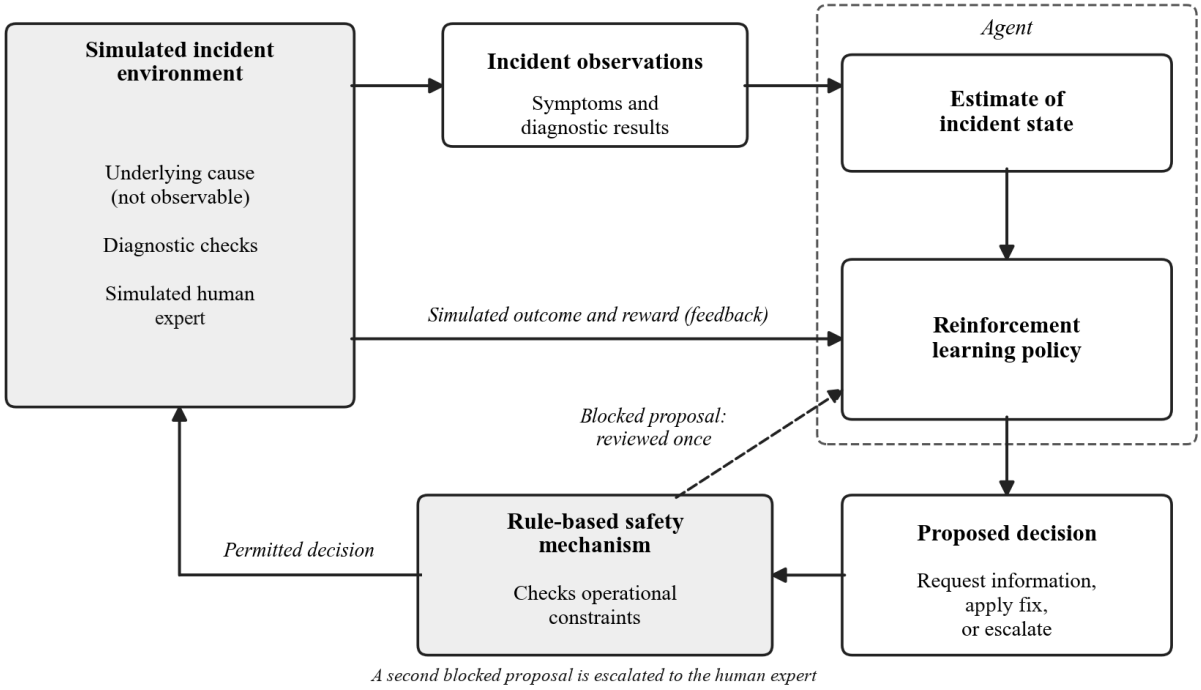


Figure 3.2: Proposed Architecture

3.6 The Reinforcement Learning Agents

Two learning algorithms are used: a deep Q-network as the primary learner and proximal policy optimisation as a secondary learner, to show whether the findings depend on the choice of algorithm. Both are taken from an established open-source library to limit implementation error [31]. Both use action masking: at step t the agent may only choose from the available actions

$$\mathcal{A}_t = \mathcal{A} \setminus \big(\{a_j^{\text{chk}} : q_{t,j} = 1\} \cup \mathcal{B}_t\big), \tag{3.15}$$

which exclude checks already requested and the set $\mathcal{B}_t$ of proposals blocked at this step.

3.6.1 Deep Q-Network:

The deep Q-network (DQN) estimates the action-value function, the expected discounted return of taking action a after observation o and following policy π afterwards [32]:

$$Q^{\pi}(o,a) = \mathbb{E}_{\pi}\left[\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \;\middle|\; o_t = o,\ a_t = a\right]. \tag{3.16}$$

The optimal action values satisfy the Bellman optimality equation, here with the maximum taken only over available actions:

$$Q^{*}(o,a) = \mathbb{E}\left[r_t + \gamma \max_{a' \in \mathcal{A}_{t+1}} Q^{*}(o_{t+1}, a') \;\middle|\; o_t = o,\ a_t = a\right]. \tag{3.17}$$

Network. Q^* is approximated by a feed-forward neural network with weights $\mathbf{w}$, which takes the encoded observation $\phi(o) \in \mathbb{R}^n$ and returns one value for each of the 36 actions. It has two hidden layers of 128 units with rectified linear (ReLU) activations, $\mathrm{ReLU}(v) = \max(0, v)$:

$$\mathbf{h}_1 = \mathrm{ReLU}\big(\mathbf{W}_1\phi(o) + \mathbf{b}_1\big), \qquad \mathbf{W}_1 \in \mathbb{R}^{128\times n}, \tag{3.18}$$

$$\mathbf{h}_2 = \mathrm{ReLU}\big(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2\big), \qquad \mathbf{W}_2 \in \mathbb{R}^{128\times 128}, \tag{3.19}$$

$$Q(o, \cdot\,; \mathbf{w}) = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3, \qquad \mathbf{W}_3 \in \mathbb{R}^{|\mathcal{A}|\times 128}, \tag{3.20}$$

where $\mathbf{w} = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{W}_3, \mathbf{b}_3\}$. A tabular method is ruled out because the check states alone admit 4^{20} combinations, and the network's output gives a value for every action, so the values the agent assigns to a blocked fix and to escalation can be inspected.

Choosing actions. During training the agent follows an ϵ-greedy policy over the available actions: with probability $1 - \epsilon$ it proposes

$$a_t = \arg\max_{a \in \mathcal{A}_t} Q(o_t, a; \mathbf{w}), \tag{3.21}$$

and with probability ϵ it proposes an available action chosen uniformly at random. The exploration rate ϵ decreases during training, and at evaluation $\epsilon = 0$.

Learning. Each transition (o_t, a_t, r_t, o_{t+1}) is stored in a replay buffer $\mathcal{E}$, from which mini-batches are sampled. The learning target uses a separate target network with weights $\mathbf{w}^-$, copied from $\mathbf{w}$ at fixed intervals, and is masked in the same way as action choice:

$$\tau_t = \begin{cases} r_t & \text{if } o_{t+1} \text{ is terminal,} \\ r_t + \gamma \max_{a' \in \mathcal{A}_{t+1}} Q(o_{t+1}, a'; \mathbf{w}^-) & \text{otherwise.} \end{cases} \tag{3.22}$$

The weights are updated by gradient descent on the loss

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}_{(o_t, a_t, r_t, o_{t+1}) \sim \mathcal{E}}\Big[\big(\tau_t - Q(o_t, a_t; \mathbf{w})\big)^2\Big], \tag{3.23}$$

for which the library uses the Huber form, which is quadratic for small errors and linear for large ones and so limits the effect of outlying targets. The full training loop, with the safety mechanism, is given as Algorithm 4.

3.6.2 Proximal Policy Optimisation:

Proximal policy optimisation (PPO) learns the policy directly rather than through action values [33]. It uses a policy network with weights $\boldsymbol{\vartheta}$ and a value network with weights $\boldsymbol{\psi}$, each with the same two hidden layers of 128 units as the DQN. The policy network outputs a score $f_a(o)$ for each action, and invalid actions are masked by giving them zero probability [34]:

$$\pi_{\boldsymbol{\vartheta}}(a \mid o_t) = \frac{\exp f_a(o_t)}{\sum_{a' \in \mathcal{A}_t} \exp f_{a'}(o_t)} \quad \text{for } a \in \mathcal{A}_t, \qquad \pi_{\boldsymbol{\vartheta}}(a \mid o_t) = 0 \text{ otherwise.} \tag{3.24}$$

The value network $V_{\boldsymbol{\psi}}(o)$ estimates the expected return from o. How much better an action was than expected is measured by the advantage, estimated with generalised advantage estimation [35]:

$$\delta_t = r_t + \gamma V_{\boldsymbol{\psi}}(o_{t+1}) - V_{\boldsymbol{\psi}}(o_t), \qquad \hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda_{\mathrm{GAE}})^l\, \delta_{t+l}, \tag{3.25}$$

where $\lambda_{\mathrm{GAE}} \in [0,1]$ trades bias against variance. PPO limits how far one update can move the policy by clipping the probability ratio between the new and the old policy:

$$\kappa_t(\boldsymbol{\vartheta}) = \frac{\pi_{\boldsymbol{\vartheta}}(a_t \mid o_t)}{\pi_{\boldsymbol{\vartheta}_{\mathrm{old}}}(a_t \mid o_t)}, \qquad \mathcal{L}^{\mathrm{CLIP}}(\boldsymbol{\vartheta}) = \mathbb{E}_t\Big[\min\big(\kappa_t \hat{A}_t,\ \mathrm{clip}(\kappa_t, 1-\epsilon_{\mathrm{clip}}, 1+\epsilon_{\mathrm{clip}})\,\hat{A}_t\big)\Big]. \quad (3.26)$$

The two networks are trained together by maximising

$$\mathcal{L}(\boldsymbol{\vartheta}, \boldsymbol{\psi}) = \mathcal{L}^{\mathrm{CLIP}}(\boldsymbol{\vartheta}) - \nu_1\, \mathbb{E}_t\Big[\big(V_{\boldsymbol{\psi}}(o_t) - \hat{G}_t\big)^2\Big] + \nu_2\, \mathbb{E}_t\Big[\mathcal{H}\big(\pi_{\boldsymbol{\vartheta}}(\cdot \mid o_t)\big)\Big], \quad (3.27)$$

where $\hat{G}_t = \hat{A}_t + V_{\boldsymbol{\psi}}(o_t)$ is the target return, $\mathcal{H}$ is the entropy of the policy, which encourages exploration, and ν_1 and ν_2 are weighting coefficients.

3.6.3 Training:

Training begins from the library's default hyperparameters, and a small search over the learning rate, the network size and the exploration schedule is carried out on seeds that are not used in the final experiments. The training length is set in pilot runs as the point at which the mean return stops improving. Each agent will be trained with ten random seeds in two regimes: at the base ambiguity level ℓ_0, and with ℓ drawn at random from its planned ranges at the start of each episode. Comparing the two regimes shows whether robustness requires exposure to ambiguity during training. Table 3.2 summarises the symbols introduced for the agents.

Table 3.2: Symbols used for the problem, the environment and the agents

Symbol	**Meaning**
Problem and environment	
$\mathcal{C}$, c_m, K	Set of causes, the m-th cause, number of causes
c	The hidden true cause of the current incident
$\mathcal{D}$, d_j, J, k_j	Set of diagnostic checks, the j-th check, number of checks, cost of check d_j
$\mathcal{X}$, x	Set of incident contexts, the context of the current incident
θ_{jm}	Probability that check d_j is positive when the cause is c_m
σ_e, E, φ_{em}	The e-th symptom (1 if present); number of symptoms; probability that symptom e is present when the cause is c_m
s_t, $\mathbf{z}$, $\mathbf{q}_t$	State at step t; vector of check results; vector of checks requested by step t
$\mathcal{A}$, a_t	Set of actions; action proposed at step t
a_j^{chk}, a_m^{fix}, a_{esc}	Request check d_j; apply the fix for cause c_m; escalate
o_t, $\mathbf{u}_t$, $\boldsymbol{\sigma}$, β_t	Observation at step t; check states; symptoms; blocked flag
$\phi(o_t)$, n	Encoded observation vector and its length
r_t, r_{correct}, λ_{wrong}, λ_{esc}, λ_{block}	Reward at step t; reward for a correct fix; penalty for a wrong fix; cost of an escalation; penalty for a blocked proposal
γ, H, T	Discount factor; step limit; length of an episode
$b_t(c_m)$, $\mathcal{I}_t$	Belief that the cause is c_m at step t; checks whose results have been observed by step t
ρ_q, N_ρ	The q-th operational rule; number of rules
$\mathcal{A}_{\mathrm{safe}}$	Actions permitted by all the rules
$a_t^{(1)}$, $a_t^{(2)}$, a_t^{exec}	First proposal, second proposal and action carried out at step t

Continued on next page

Symbol	Meaning
$\mathcal{A}_t$, $\mathcal{B}_t$	Available actions at step t; proposals blocked at step t
$\eta(\pi)$	Expected discounted return of policy π
p_{miss}, p_{noise}, ω	Probabilities that a check is missing or reversed; overlap between causes in a category
$\tilde{\theta}_{jm}$, θ'_{jm}, φ'_{em}, $g(m)$	Check likelihood after noise; check and symptom likelihoods after overlap; category of cause c_m
y	Reference label: 1 if the incident should be escalated
Deep Q-network	
$Q^{\pi}(o,a)$, Q^{*}	Action value under policy π; optimal action value
$\mathbf{w}$, $\mathbf{w}^{-}$	Weights of the Q-network and of the target network
$\mathbf{W}.$, $\mathbf{b}.$, $\mathbf{h}.$	Weight matrices, bias vectors and hidden-layer outputs
ϵ	Exploration rate of the ϵ-greedy policy
$\mathcal{E}$, τ_t, $\mathcal{L}$	Replay buffer; learning target; loss
Proximal policy optimisation	
$\boldsymbol{\vartheta}$, $\boldsymbol{\psi}$	Weights of the policy network and of the value network
$f_a(o)$, $\pi_{\boldsymbol{\vartheta}}$, $V_{\boldsymbol{\psi}}$	Score of action a; policy; value estimate
δ_t, $\hat{A}_t$, λ_{GAE}	Temporal-difference error; estimated advantage; advantage-estimation parameter
κ_t, ϵ_{clip}	Probability ratio between new and old policy; clipping range
$\hat{G}_t$, $\mathcal{H}$, ν_1, ν_2	Target return; policy entropy; loss weights

3.7 Baseline Decision Policies

The agent will be compared with six baseline policies that run in the same environment on the same evaluation incidents, each standing for an approach from Chapter 2 (Table 3.3).

Table 3.3: Baseline decision policies

Policy	Behaviour	Approach represented
Initial-symptom	Applies the fix for the cause that the prior and the symptoms make most likely (Equation (3.8) at $t = 0$), without checks	Triage as one prediction
Scripted checklist	Requests a fixed sequence of checks for the category, elicited from Directorate practice, then fixes the most probable cause	Runbook practice
Always-escalate	Escalates every incident	The limit of deferral
Confidence threshold	Tracks the belief b_t of Equation (3.8), requests the most informative check until one cause passes a threshold, else escalates	Uncertainty-driven investigation (knows the model)
Unconstrained agent	The same DQN without the safety mechanism	Standard shielding comparison
Penalty-only agent	The same DQN without the mechanism, with a reward penalty per rule violation	Constraint by reward penalty

The confidence-threshold policy is given as Algorithm 5. Two ablations of the agent, one that cannot request checks and one that cannot escalate, will show what each option contributes.

3.8 Prototype: the Agent as a Decision Layer

To show how the trained agent would sit in a support workflow, a prototype places it between an incident ticket and the action taken on it. A reader turns the ticket and its logs into the symptoms, context and check results that the agent observes. It is either a keyword reader that runs offline or a large language model that returns a fixed structured form, in which every result must be supported by a quotation that is verified to appear in the text, and the text is redacted before it is sent. A check that needs a full analysis of the logs can be delegated by the agent to a second language-model call, which is treated as one more noisy check in the scenario model, so that the agent learns when delegation is worth its cost. The agent then plays the incident under the safety mechanism, and its decision is carried out in one of three ways: a fix for a fault in code becomes a draft pull request on a fictional repository, accepted only if the repository's tests pass; any other fix becomes a fix order to the owner of that cause; and an escalation is routed to the owner of the most probable cause, or to a senior developer when no cause is probable enough, together with the evidence gathered. An agent trained with verification has each fix verified before it goes on, as in the simulation: a fix for a fault in code is applied to a staging copy of the repository and passes only if the test that reproduces the incident now passes and no other test breaks, and any other fix is confirmed by the operator. A fix that fails is reported to the agent, which continues to investigate, tries another fix or escalates. The final decision is checked against the rules a second time before anything is done, and the prototype never merges a change or acts on a live system. The prototype is not part of the hypotheses; the error rates of its reader on sample tickets written by Directorate staff are reported because they indicate realistic values of p_{noise} and p_{miss}.

3.9 Evaluation Strategy

The framework will be evaluated across four groups of measures: decision quality, investigation efficiency, escalation behaviour and constraint compliance. At each level of ambiguity, $S = 10$ evaluation sets of $N = 1{,}000$ incidents each are generated from seeds never used in training, and every policy is run on the same sets. Set s is paired with training seed s of each learning agent; a baseline that does not learn is run on all S sets, so that every policy has S values of each measure. With $N = 1{,}000$, the standard error of a rate measured on one set is at most $\sqrt{0.25/1000} \approx 0.016$, small enough to resolve the differences of 0.05 used in the decision criteria (Section 3.9.7). Each episode i ends in one of three outcomes o_i: a correct fix (c), a wrong fix (w) or an escalation (e). Let N_{c}, N_{w} and N_{e} be the numbers of episodes with each outcome, so that

$$N = N_{\text{c}} + N_{\text{w}} + N_{\text{e}}. \tag{3.28}$$

Escalations include those chosen by the agent, those forced by the safety mechanism and those caused by the step limit. Table 3.4 defines the symbols used in the rest of this section.

Table 3.4: Symbols used in the evaluation measures

Symbol	**Meaning**
Episodes and outcomes	
N	Number of incidents in one evaluation set ($N = 1{,}000$); each episode is the triage of one incident
i	Index of an evaluation episode, $i = 1, \ldots, N$
o_i	Outcome of episode i: c (correct fix), w (wrong fix) or e (escalation)
$N_{\text{c}}, N_{\text{w}}, N_{\text{e}}$	Numbers of episodes ending in a correct fix, a wrong fix and an escalation

Continued on next page

Symbol	Meaning
T_i	Number of steps taken in episode i
$r_{i,t}$	Reward received at step t of episode i
π	A decision policy: the proposed agent or one of the baselines
Investigation	
K_i	Set of diagnostic checks requested in episode i; $\lvert K_i \rvert$ is their number
k_j	Cost of diagnostic check j
Escalation	
y_i	Reference label of episode i: 1 if the incident should be escalated, 0 otherwise
$\hat{y}_i$	Agent's choice in episode i: 1 if it chose to escalate, 0 otherwise
TP, FP, FN, TN	True positives, false positives, false negatives and true negatives: the four cells of the confusion matrix of $\hat{y}_i$ against y_i
Constraint compliance	
M	Total number of actions proposed by the agent over the evaluation episodes
M_{b}	Number of proposed actions blocked by the safety mechanism
N_{f}	Number of episodes ending in a forced escalation after a second blocked proposal
N_{v}	Number of episodes in which a prohibited action took effect
v_i	1 if a prohibited action took effect in episode i, 0 otherwise
$\mathbf{1}[\cdot]$	Indicator function: 1 when the condition in brackets holds, 0 otherwise
Robustness and reporting	
ℓ	Level of ambiguity, set by p_{miss}, p_{noise} and ω (Section 3.4)
ℓ_0	Base level of ambiguity, at which the agent is trained in the first regime
S	Number of training seeds per learning agent, and of evaluation sets per ambiguity level ($S = 10$)
s	Index of a seed, $s = 1, \ldots, S$
X, X_s, $\bar{X}$	Any measure; its value on seed s; its mean over seeds
B	Number of bootstrap resamples used for the confidence interval ($B = 10{,}000$)
Measures	
Acc	Resolution accuracy
Cov	Coverage: share of incidents on which the agent applies a fix
SelAcc	Selective accuracy: accuracy of the fixes applied
WR	Wrong-fix rate
$\bar{G}$	Mean episodic return
$\bar{n}$, $\bar{C}$	Mean number of checks and mean investigation cost per episode
ER	Escalation rate
P, R, F_1	Escalation precision, recall and F1 score
MCC	Matthews correlation coefficient of escalations
PVR	Proposed-violation rate
FER	Forced-escalation rate
EVR	Executed-violation rate
CSR	Compliant success rate
$\Delta\mathrm{Acc}(\ell)$	Change in resolution accuracy at level ℓ relative to ℓ_0
$\mathbf{m}(\ell)$	Decision mix at level ℓ: coverage, escalation rate and mean number of checks

3.9.1 Decision Quality:

Decision quality describes how often the final decision is right. Resolution accuracy is the share of all incidents that the agent resolves itself, the usual accuracy of the triage literature [2]:

$$\mathrm{Acc} = \frac{N_{\mathrm{c}}}{N}. \tag{3.29}$$

Because an agent can avoid errors by escalating, accuracy is read together with coverage and selective accuracy, the standard pair of selective prediction [22]:

$$\mathrm{Cov} = \frac{N_{\mathrm{c}} + N_{\mathrm{w}}}{N}, \qquad \mathrm{SelAcc} = \frac{N_{\mathrm{c}}}{N_{\mathrm{c}} + N_{\mathrm{w}}}. \tag{3.30}$$

The wrong-fix rate isolates the most costly error in triage:

$$\mathrm{WR} = \frac{N_{\mathrm{w}}}{N}. \tag{3.31}$$

The mean episodic return is the quantity the agent is trained to maximise [3]:

$$\bar{G} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} r_{i,t}, \tag{3.32}$$

where T_i is the number of steps in episode i and $r_{i,t}$ is the reward at step t. Because $\bar{G}$ depends on the reward values chosen, it is never reported alone.

3.9.2 Investigation Efficiency:

Investigation efficiency describes what the agent spends on evidence before it decides. Let K_i be the set of checks requested in episode i and k_j the cost of check j. The mean number of checks and the mean investigation cost are

$$\bar{n} = \frac{1}{N} \sum_{i=1}^{N} |K_i|, \qquad \bar{C} = \frac{1}{N} \sum_{i=1}^{N} \sum_{j \in K_i} k_j. \tag{3.33}$$

The cost stands in for time, which a simulation without a clock cannot measure directly. Following the evaluation of costly feature acquisition [23], each policy π is also plotted as a point $(\bar{C}_\pi, \mathrm{Acc}_\pi)$ on an accuracy–cost curve, which shows whether additional investigation buys additional accuracy and allows policies to be compared at equal cost.

3.9.3 Escalation Behaviour:

Escalation behaviour describes how much the agent hands to the human expert and whether it hands over the right cases. The escalation rate is the load placed on human experts [7]:

$$\mathrm{ER} = \frac{N_{\mathrm{e}}}{N}. \tag{3.34}$$

The remaining measures compare the agent's own choices with the reference labels. For each episode, let $y_i = 1$ if the incident is labelled as one that should be escalated and $y_i = 0$ otherwise, and let $\hat{y}_i = 1$ if the agent chose to escalate and $\hat{y}_i = 0$ otherwise. Forced escalations and step-limit escalations are excluded, because the agent did not choose them; they are reported under constraint compliance. Over the remaining episodes,

$$\mathrm{TP} = \sum_i \hat{y}_i\, y_i, \quad \mathrm{FP} = \sum_i \hat{y}_i (1 - y_i), \quad \mathrm{FN} = \sum_{23^i} (1 - \hat{y}_i)\, y_i, \quad \mathrm{TN} = \sum_i (1 - \hat{y}_i)(1 - y_i). \tag{3.35}$$

Escalation precision is the share of the agent's escalations that should have been escalated; low precision means that human experts receive incidents the agent could have resolved. Escalation recall is the share of incidents that should have been escalated that the agent did escalate; low recall means the agent kept incidents it should have handed over. The F1 score is their harmonic mean:

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad F_1 = \frac{2PR}{P + R}. \tag{3.36}$$

Because precision, recall and F1 can all look acceptable when one class is rare, the Matthews correlation coefficient, which uses all four cells of the confusion matrix and exposed degenerate triage policies that F1 concealed [26], is also reported:

$$\mathrm{MCC} = \frac{\mathrm{TP} \cdot \mathrm{TN} - \mathrm{FP} \cdot \mathrm{FN}}{\sqrt{(\mathrm{TP} + \mathrm{FP})(\mathrm{TP} + \mathrm{FN})(\mathrm{TN} + \mathrm{FP})(\mathrm{TN} + \mathrm{FN})}}. \tag{3.37}$$

MCC ranges from -1 to $+1$, where 0 means the agent's escalations are no better than chance.

3.9.4 Constraint Compliance:

Constraint compliance describes the agent's relationship to the rules. Let M be the total number of actions the agent proposes over the evaluation episodes and M_{b} the number blocked by the safety mechanism. The proposed-violation rate shows whether the agent has learned to act within the rules or relies on being stopped:

$$\mathrm{PVR} = \frac{M_{\mathrm{b}}}{M}. \tag{3.38}$$

Let N_{f} be the number of episodes that end in a forced escalation after a second blocked proposal, and N_{v} the number in which a prohibited action takes effect. The forced-escalation rate and the executed-violation rate are

$$\mathrm{FER} = \frac{N_{\mathrm{f}}}{N}, \qquad \mathrm{EVR} = \frac{N_{\mathrm{v}}}{N}. \tag{3.39}$$

The executed-violation rate is zero by construction when the mechanism is active, and it is the central measure for the unconstrained and penalty-only agents, as in the shielding literature [14]. Let $v_i = 1$ if a prohibited action took effect in episode i. The compliant success rate, which follows the measure of task completion under policy used for agent benchmarks [36], is

$$\mathrm{CSR} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left[o_i = \mathrm{c} \ \wedge \ v_i = 0\right], \tag{3.40}$$

where $\mathbf{1}[\cdot]$ equals 1 when its condition holds and 0 otherwise. For agents without the mechanism, the gap between Acc and CSR shows how much of their accuracy was obtained by breaking rules.

3.9.5 Robustness:

Robustness is not a separate metric: every measure above is computed at each ambiguity level ℓ and reported as a function of it. Degradation is the change relative to the base level ℓ_0, for example

$$\Delta\mathrm{Acc}(\ell) = \mathrm{Acc}(\ell) - \mathrm{Acc}(\ell_0), \tag{3.41}$$

and likewise for the wrong-fix rate. The decision mix at each level,

$$\mathbf{m}(\ell) = \big(\mathrm{Cov}(\ell),\ \mathrm{ER}(\ell),\ \bar{n}(\ell)\big), \tag{3.42}$$

shows whether the agent responds to ambiguity by investigating more, by escalating more, or by continuing to fix on weaker evidence, which is the behaviour tested by hypotheses H3a to H3c.

3.9.6 Reporting:

Each learning agent is trained with $S = 10$ random seeds, and each seed is evaluated on its paired evaluation set. For any measure X, the reported value is the mean over the S values,

$$\bar{X} = \frac{1}{S}\sum_{s=1}^{S} X_s, \tag{3.43}$$

with a 95% percentile bootstrap confidence interval obtained by resampling the S values with replacement $B = 10{,}000$ times and taking the 2.5th and 97.5th percentiles of the resampled means [37].

3.9.7 Decision Criteria:

The hypotheses are judged by the following criteria, fixed before any experiment is run. A policy is said to score *higher* than another on a rate when its mean exceeds the other's by at least 0.05 and the two 95% bootstrap confidence intervals do not overlap; *lower* is defined in the same way. For the mean number of checks, the minimum difference is 0.5 checks. Any other outcome counts as no difference.

- **H1** is supported if, at ℓ_0, the agent's resolution accuracy is higher than that of each of the initial-symptom, scripted-checklist and always-escalate policies, and its mean investigation cost is not higher than that of the scripted checklist. Otherwise it is not supported.
- **H2** is supported if, at ℓ_0, the agent's compliant success rate is higher than that of both the unconstrained agent and the penalty-only agent. The executed-violation rate of the agent with the mechanism is checked separately as a verification of the implementation: it must be zero on every evaluation incident, and any other value is treated as a defect to be corrected before results are reported.
- **H3a to H3c** are judged one factor at a time, by comparing the agent at ℓ_0 with the agent at the highest planned value of the factor concerned, the other two factors held at their base values. H3a is supported if, as p_{miss} rises, the escalation rate is higher and the mean number of checks is not higher. H3b is supported if, as p_{noise} rises, the mean number of checks is higher and coverage is lower. H3c is supported if, as ω rises, the mean number of checks and the escalation rate are both higher.

Each hypothesis is reported as supported or not supported, whichever the results show.

3.9.8 Experiments:

Four experiments use these measures. Experiment 1 compares the agent with the six baselines at the base level of ambiguity, and is repeated with the PPO agent to check that the findings do not depend on the learning algorithm. Experiment 2 compares the agent with the unconstrained

and penalty-only agents and tracks the proposed-violation rate over training. Experiment 3 compares it with its two ablations. Experiment 4 evaluates every policy across ambiguity levels, for both training regimes. Its sweep is one factor at a time from ℓ_0: each control takes its three non-base planned values while the other two stay at their base values, giving nine levels, which with ℓ_0 make ten. Two combined levels are added, with all three controls at their second-highest and at their highest planned values, giving twelve ambiguity settings in all. A sensitivity analysis repeats Experiment 1 with the escalation cost set to $\lambda_{\text{esc}} \in \{1, 3, 6, 9\}$, all below λ_{wrong}. All measures are computed on the evaluation sets described at the start of this section, following Algorithm 6.

3.10 Limitations

The design has limitations that bound what its results can show.

- **Simulation and reality.** The agent is trained and evaluated only in a simulator built from staff ratings, so the results show how it behaves in that model, not how it would perform at the Directorate.
- **An expert who always succeeds.** The simulated human expert resolves every escalated incident, so escalation is never itself a source of error, and its only cost is λ_{esc}.
- **A low cost of escalation.** With $\lambda_{\text{esc}} = 3$ against $\lambda_{\text{wrong}} = 10$, escalation is cheap relative to a wrong fix, which may understate its burden on experts and encourage escalation; the sensitivity analysis over $\lambda_{\text{esc}} \in \{1, 3, 6, 9\}$ shows how far the conclusions depend on this value.
- **Conditional independence.** Symptoms and check results are assumed to be independent given the cause. In practice, checks that read the same logs may be correlated, which would make evidence less informative than the simulator assumes.
- **Researcher-chosen rewards.** The reward values are set by the researcher and tuned in pilot runs, not measured at the Directorate, and the agent's behaviour depends on them.
- **Reliable verification.** In the variant with verification, a fix that clears the symptoms is assumed to have removed the true cause, and the remedy times and harms are set by the researcher until the Directorate confirms them.
- **Actions in the prototype.** The prototype acts only on a fictional repository and through simulated messages; it shows how decisions would be carried out, not that they are safe to carry out on live systems.
- **One institution.** The scenario model comes from a single IT directorate, so its causes, checks, rules and ratings may not represent other organisations.

Chapter 4

Expected Results

4.1 Decision Quality

At the base level of ambiguity, the agent is expected to resolve a larger share of incidents correctly than the initial-symptom, scripted-checklist and always-escalate policies, with a mean investigation cost no higher than that of the scripted checklist (hypothesis H1). The hypothesis is judged by the criteria of Section 3.9.7: a difference of at least 0.05 in resolution accuracy with non-overlapping 95% bootstrap confidence intervals. The agent is also expected to approach the confidence-threshold policy, which is given the likelihood model that the agent must learn from experience; the gap between the two is reported but is not part of H1. The results will be reported whether or not the hypothesis holds.

4.2 Rule Compliance

The agent with the safety mechanism is expected to achieve a higher compliant success rate than the unconstrained and penalty-only agents (hypothesis H2), by at least 0.05 with non-overlapping confidence intervals. Because a blocked proposal earns the block penalty λ_{block}, the proposed-violation rate is expected to fall over training, which would show that the agent learns to act within the rules and does not rely on being stopped.

That no prohibited action is carried out while the mechanism is active is not part of H2, because it holds by construction. It is verified instead: the executed-violation rate of the agent with the mechanism must be zero on every evaluation incident.

4.3 Robustness to Ambiguity

The agent is expected to respond differently to each source of ambiguity, judged one factor at a time from the base level:

- **Missing evidence (H3a):** as p_{miss} rises, the agent is expected to escalate more incidents without requesting more checks, since a missing check cannot be requested and there is less evidence left to gather.
- **Unreliable evidence (H3b):** as p_{noise} rises, the agent is expected to request more checks before deciding, to corroborate results it can no longer trust, and to apply fixes on fewer incidents.

- **Overlapping evidence (H3c):** as ω rises, the agent is expected to request more checks and to escalate more incidents, since causes in the same category become harder to tell apart.

The agent trained across randomised ambiguity levels is expected to degrade less than the agent trained at a single level. The experiments will report what no reviewed study has reported: how the shares of investigating, fixing and escalating change across controlled levels of missing, unreliable and overlapping evidence.

4.4 Pilot Results

Before the Directorate elicitation, the full design was run on a pilot scenario model of 8 causes, 11 checks and 5 rules supplied by one developer of the Directorate, with fix with verification enabled and three training seeds per agent. The pilot shows that the method runs, learns and can be evaluated as planned; its results are not findings of the study. At the elicited cost of escalation, the learned agents reached the confidence-threshold policy, which is given the likelihood model, and were well ahead of the scripted, initial-symptom and always-escalate policies on return. The safety mechanism kept executed violations at zero, against 7.6% of incidents for the unconstrained agent. The pilot also exposed two points of the design that the main study will address. Without a cost for a blocked proposal, the agent behind the mechanism continued to propose prohibited fixes in about a fifth of its proposals, so a block penalty $\lambda_{\text{block}} > 0$ will be used as specified. With verification, a checklist that tries one fix after another resolved the most incidents at a low return, so resolution accuracy will be read together with return when hypothesis H1 is judged.

4.5 Prototype

The research is expected to produce a working simulator of IT incident triage built from the Directorate's scenarios, the rule-based safety mechanism, six baseline policies, trained agents, and scripts that reproduce every experiment. Two dashboards accompany them. A training dashboard follows each run and replays an incident step by step, showing the evidence revealed, the agent's estimated action values, any blocked proposal and the final decision. A prototype dashboard runs the decision layer of Section 3.8 end to end: a ticket and its logs are read, the agent investigates, fixes or escalates under the safety mechanism, and the decision is carried out as a draft pull request on a fictional repository, a fix order, or an escalation routed to the owner of the most probable cause, with every step and its evidence shown.

Broader impact. Putting this together, the expected results address the problem this research is concerned with: that automated triage tools predict an answer for every incident and do not know when to look further, when to act within the rules, and when to hand the incident to a person. An agent that makes these three decisions under explicit, readable rules, evaluated on all four groups of measures, would give IT support teams such as the Directorate evidence on how far such automation can be trusted before it is considered for use.

Bibliography

[1] Z. Chen *et al.*, "Towards intelligent incident management: Why we need it and how we make it," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2020, pp. 1487–1497. [Online]. Available: https://doi.org/10.1145/3368089.3417055

[2] Y. Remil, A. Bendimerad, R. Mathonat, and M. Kaytoue, "AIOps solutions for incident management: Technical guidelines and a comprehensive literature review," arXiv preprint arXiv:2404.01363, 2024. [Online]. Available: https://arxiv.org/abs/2404.01363

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998. [Online]. Available: https://doi.org/10.1016/S0004-3702(98)00023-X

[5] M. Alauthman, A. Almomani, A. Al-Qerem, G. Albesani, and M. Alkasassbeh, "Explainable reinforcement learning for automated incident response in critical infrastructure," in *Proc. 2nd Int. Conf. Comput. Intell. Approaches Appl. (ICCIAA)*, 2026, pp. 1–7. [Online]. Available: https://doi.org/10.1109/ICCIAA68481.2026.11543556

[6] V. Kovtun, A. Huralnyk, L. Husak, and O. Kovaliuk, "Risk-sensitive decision-making in security operations center alert escalation under resource constraints," in *CEUR Workshop Proceedings*, vol. 4242, 2026, pp. 65–83.

[7] E. Turan, "Oversight has a capacity: Calibrating agent guards to a subjective, fatiguing human," arXiv preprint arXiv:2606.08919, 2026. [Online]. Available: https://arxiv.org/abs/2606.08919

[8] D. Dunsin, M. C. Ghanem, K. Ouazzane, and V. Vassilev, "Reinforcement learning for an efficient and effective malware investigation during cyber incident response," *High-Confidence Computing*, vol. 5, no. 3, p. 100299, 2025. [Online]. Available: https://doi.org/10.1016/j.hcc.2025.100299

[9] K. Hammar and T. Li, "Online incident response planning under model misspecification through Bayesian learning and belief quantization," in *Proc. 18th ACM Workshop Artif. Intell. Secur. (AISec)*, 2025, pp. 40–51. [Online]. Available: https://doi.org/10.1145/3733799.3762965

[10] K. Hammar, Y. Li, T. Alpcan, E. C. Lupu, and D. P. Bertsekas, "Adaptive network security policies via belief aggregation and rollout," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 6544–6559, 2026. [Online]. Available: https://doi.org/10.1109/TIFS.2026.3709131

[11] F. Şahin and N. S. Mert, "Dual-Threshold Conformal Deferral for Trustworthy Security Alert Triage," *Electronics (Switzerland)*, vol. 15, no. 18, 2026, art. no. 4084. [Online]. Available: https://doi.org/10.3390/electronics15184084

[12] B. Könighofer, R. Bloem, N. Jansen, S. Junges, and S. Pranger, "Shields for safe reinforcement learning," *Communications of the ACM*, vol. 68, no. 11, pp. 80–90, 2025.

[Online]. Available: https://doi.org/10.1145/3715958

[13] L. Zhang *et al.*, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–35, 2025, art. no. 44. [Online]. Available: https://doi.org/10.1145/3746635

[14] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018. [Online]. Available: https://doi.org/10.1609/aaai.v32i1.11797

[15] M. J. Page, J. E. McKenzie, P. M. Bossuyt *et al.*, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.

[16] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.

[17] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 70, 2017, pp. 22–31.

[18] W. Zhao, T. He, R. Chen, T. Wei, and C. Liu, "State-wise safe reinforcement learning: A survey," in *Proc. 32nd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2023, pp. 6814–6822. [Online]. Available: https://doi.org/10.24963/ijcai.2023/763

[19] W.-C. Yang, G. Marra, G. Rens, and L. De Raedt, "Safe reinforcement learning via probabilistic logic shields," arXiv preprint arXiv:2303.03226, 2023. [Online]. Available: https://arxiv.org/abs/2303.03226

[20] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[21] S. Joshi, S. Parbhoo, and F. Doshi-Velez, "Learning-to-defer for sequential medical decision-making under uncertainty," *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://openreview.net/forum?id=0pn3KnbH5F

[22] Y. Geifman and R. El-Yaniv, "SelectiveNet: A deep neural network with an integrated reject option," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 97, 2019, pp. 2151–2159.

[23] J. Janisch, T. Pevný, and V. Lisý, "Classification with costly features using deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 3959–3966. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33013959

[24] L. Wang *et al.*, "Root cause analysis for microservice systems via hierarchical reinforcement learning from human feedback," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD)*, 2023, pp. 5116–5125. [Online]. Available: https://doi.org/10.1145/3580305.3599934

[25] R. Ding *et al.*, "TraceDiag: Adaptive, interpretable, and efficient root cause analysis on large-scale microservice systems," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2023, pp. 1762–1773. [Online]. Available: https://doi.org/10.1145/3611643.3613864

[26] E. Okafor, "A leakage-free benchmark of offline policy learning for SOC alert triage," *Machine Learning with Applications*, vol. 25, 2026, art. no. 100984. [Online]. Available: https://doi.org/10.1016/j.mlwa.2026.100984

[27] R. V. Ravi, K. Priyanga, and M. Adarsh, "SAFE-HGX: A Hierarchical and Safety-Aware Reinforcement Learning Framework for Intrusion Response in Industrial Control Systems," in *2025 1st International Conference on Advancement in Futuristic Technologies, ICAFT 2025*, 2025. [Online]. Available: https://doi.org/10.1109/ICAFT66710.2025.11453104

[28] T. Singh, "Autonomous AI Decision Control Tower: A Deterministic Multi-Agent Decision Intelligence Framework for Enterprise NoOps," in *2026 IEEE 1st International*

Conference on Artificial Intelligence Implementation and Applications, ICAIIA 2026, 2026, pp. 190–195. [Online]. Available: https://doi.org/10.1109/ICAIIA70172.2026.11691884

[29] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–106, 2004. [Online]. Available: https://doi.org/10.2307/25148625

[30] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv preprint arXiv:2407.17032, 2024. [Online]. Available: https://arxiv.org/abs/2407.17032

[31] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[32] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: https://doi.org/10.1038/nature14236

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[34] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. 35th Int. Florida Artif. Intell. Res. Soc. Conf. (FLAIRS)*, 2022. [Online]. Available: https://arxiv.org/abs/2006.14171

[35] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. 4th Int. Conf. Learn. Represent. (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1506.02438

[36] I. Levy *et al.*, "ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2410.06703

[37] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[38] National Planning Authority, "Fourth National Development Plan (NDP IV) 2025/26–2029/30," National Planning Authority, Kampala, Uganda, Tech. Rep., 2025.

[39] National Planning Authority, "Uganda Vision 2040," National Planning Authority, Kampala, Uganda, Tech. Rep., 2013. [Online]. Available: https://npa.go.ug/wp-content/uploads/2023/03/VISION-2040.pdf

Appendix A

Budget

The research has few direct costs. Training and evaluation will run on the university's computing facilities and the researcher's laptop with open-source software (Python, Gymnasium, Stable-Baselines3 and PyTorch), and the elicitation sessions with the Directorate of University ICT Services involve no cost. The only expected cost is printing. Unit costs are estimates based on the author's earlier proposal budget.

No.	Item	Details	Unit Cost (UGX)	Qty	Total (UGX)
1	Project report printing and binding	3 copies for the supervisor, department, personal	50,000	3	150,000
2	Proposal printing	2 copies for supervisor review	10,000	2	20,000
				TOTAL BUDGET (UGX)	**170,000**

Appendix B

Research Project timelines

The project is managed in seven phases. The implementation runs over six weeks, from 19 October to 29 November 2026, after the literature review and the stakeholder interview. Each phase depends on the one before it: the environment cannot be built before the scenarios are elicited, the scenario model is frozen before any agent is trained, and the experiments cannot run before the agents and baselines exist.

The seven phases group the eight research phases of Figure 3.1 for scheduling. Phase 1 covers specifying the problem; Phase 2, eliciting scenarios with the Directorate; Phase 3, both building and validating the environment and encoding the rules; Phase 4, both implementing the baselines and training the agents; Phase 5, running the experiments; and Phase 6, analysing the results. Phase 7, writing and submission, is not a research phase and so does not appear in Figure 3.1.

Project Timeline (19 October to 29 November 2026)

Phase	Phase Name	Key Activities	Weeks	Depends on	Deliverable
1	Literature and Planning	Thematic literature review and Chapters 1–3; stakeholder interview with the Directorate (19 Oct); systematic literature review (23 Oct); approvals; elicitation workbook.	Before 19 Oct, Week 1	–	Proposal, systematic review, elicitation workbook.
2	Scenario Elicitation	Elicitation session(s) with Directorate staff in the week of 26 October; scenario model of about 15 causes, about 10 symptoms, 20 checks and 10 rules written up as configuration files.	Week 2	1	Scenario model v1.

Continued on next page

Phase	Phase Name	Key Activities	Weeks	Depends on	Deliverable
3	Environment and Rules	Incident generator and Gymnasium environment with the three ambiguity controls; the rules and the review-then-escalate mechanism; tests and leakage check; staff review of sample incidents; scenario model frozen at the end of the week.	Week 3	2	Frozen scenario model; tested simulator and rule mechanism.
4	Baselines and Training	Six baseline policies and the metrics module; pilot runs; small hyperparameter search; the agent, unconstrained and penalty-only agents and the PPO agent trained over ten seeds on the frozen model.	Week 4	3	Baselines run end to end; trained agents.
5	Experiments	Experiments 1–3 (baselines, effect of the mechanism, ablations); sensitivity to escalation cost; Experiment 4 (twelve ambiguity settings).	Week 5	4	Results tables and charts.
6	Analysis and Prototype	Confidence intervals, decision-mix charts and findings against the decision criteria for H1–H3; replay dashboard and results panel.	Weeks 5–6	5	Findings summary; working prototype.
7	Writing and Submission	Results chapter, revision of Chapters 1–3, poster, archive of code and configurations, presentation rehearsal.	Week 6	6	Poster, report, archived code.

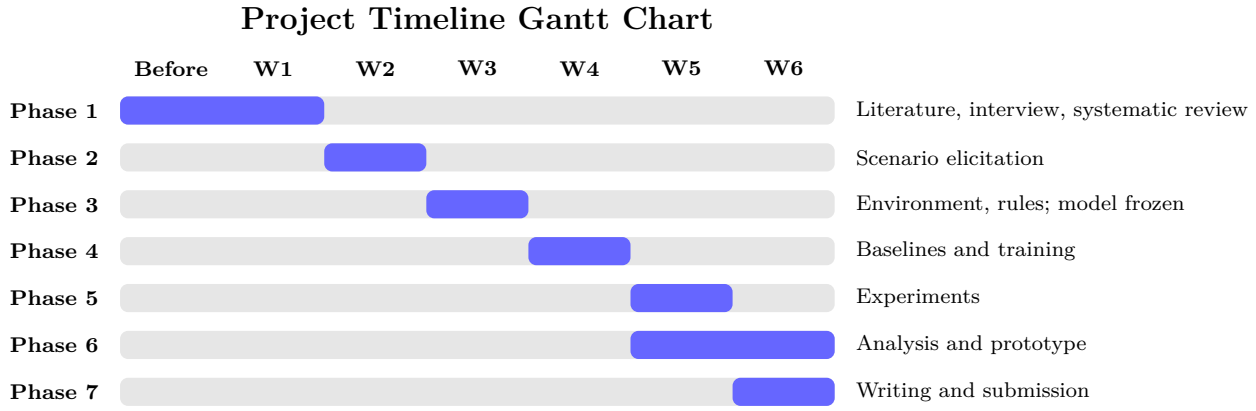


Figure B.1: Research project timeline Gantt chart. W1 begins on 19 October 2026 and W6 ends on 29 November 2026. The scenario model is frozen at the end of W3, before training begins in W4.

Appendix C

Declaration of AI use

Declaration Statement:

I, *OPOL CHARIS*, declare that I used

i. ***Claude by Anthropic (Claude Opus 5.5)*** as a research assistant and sparring partner throughout the development of this research. I used it to review my arguments, identify weaknesses in my reasoning, and suggest improvements to the clarity and structure of my work. I also used it to assist with the systematic literature review, including refining search strategies, reviewing literature, and strengthening the discussion of research gaps. I evaluated the suggestions and retained or modified them based on my own research and understanding.

ii. ***Claude by Anthropic*** to assist with the technical presentation of my research, including writing and formatting LaTeX code for tables, generating some figures, and improving document layout. I reviewed the generated code and outputs and made the necessary modifications to ensure that they accurately represented my work.

Signature: _____ Date: _____

Appendix D

List of 10 Major Publications Reviewed

The ten publications below are listed in order of their relevance to this research. They comprise four journal articles, three conference papers and three reviews, one of which, Remil et al., is an arXiv preprint. Impact is given as the Field-Weighted Citation Impact (FWCI) reported by Scopus, which compares a publication's citations with the average for publications of the same field, type and year; a value of 1.00 is the world average. The first three publications have no FWCI yet because they were published in 2026; they are included because they appeared in peer-reviewed venues and address the research problem most directly. The Remil et al. review is not indexed in Scopus.

Publications Ranked by Relevance

Rank	Title	Journal / Venue	Type	FWCI
1	Sahin & Mert (2026): Dual-Threshold Conformal Deferral for Trustworthy Security Alert Triage [11]	Electronics	Journal Article	N/A (2026)
2	Kovtun et al. (2026): Risk-Sensitive Decision-Making in SOC Alert Escalation under Resource Constraints [6]	CEUR Workshop Proceedings	Conference Paper	N/A (2026)
3	Okafor (2026): A Leakage-Free Benchmark of Offline Policy Learning for SOC Alert Triage [26]	Machine Learning with Applications	Journal Article	N/A (2026)
4	Hammar et al. (2026): Adaptive Network Security Policies via Belief Aggregation and Rollout [10]	IEEE Transactions on Information Forensics and Security	Journal Article	2.91
5	Chen et al. (2020): Towards Intelligent Incident Management [1]	ESEC/FSE	Conference Paper	3.42
6	Remil et al. (2024): AIOps Solutions for Incident Management [2]	arXiv	Review	Not in Scopus

Continued on next page

Rank	Title	Journal / Venue	Type	FWCI
7	Alshiekh et al. (2018): Safe Reinforcement Learning via Shielding [14]	AAAI Conference on Artificial Intelligence	Conference Paper	22.46
8	Könighofer et al. (2025): Shields for Safe Reinforcement Learning [12]	Communications of the ACM	Review	2.82
9	García & Fernández (2015): A Comprehensive Survey on Safe Reinforcement Learning [16]	Journal of Machine Learning Research	Review	4.23
10	Mnih et al. (2015): Human-Level Control through Deep Reinforcement Learning [32]	Nature	Journal Article	151.38

Relevance Key

	Addresses reinforcement learning or sequential decisions in IT or security incident handling
	Provides the methods the study builds on: safe reinforcement learning, shielding, deep Q-networks

Appendix E

Scopus Search Strings used during the literature review

The systematic review searched three databases on 23 October 2026. The strings are given exactly as they were run. They exported 3,394 records from Scopus, 1,000 from the ACM Digital Library and 5 from IEEE Xplore.

E.1 Scopus and ACM Digital Library (all fields)

```
( "reinforcement learning" OR "deep Q-network*" OR "Q-learning" OR "policy gradient" OR
"actor-critic" OR "proximal policy optimi*" OR "Markov decision process*" OR "partially
observable Markov" OR POMDP OR "contextual bandit*" OR "multi-armed bandit*" ) AND (
"incident management" OR "incident response" OR "incident triage" OR "incident handling" OR
"incident resolution" OR "incident mitigation" OR AIOps OR "IT operations" OR "IT service
management" OR ITSM OR "site reliability" OR "service desk" OR "help desk" OR "trouble
ticket*" OR "ticket routing" OR "ticket assignment" OR "alert triage" OR "alert prioriti*"
OR "security operations cent*" OR "automated remediation" OR ( "root cause" AND (
microservice* OR cloud OR "online service*" OR "IT system*" ) ) ) AND ( triage OR escalat*
OR defer* OR "human-in-the-loop" OR "human in the loop" OR "human oversight" OR investigat*
OR diagnos* OR "information gathering" OR "information acquisition" OR remediat* OR mitigat*
OR constrain* OR shield* OR "safe reinforcement learning" OR "action mask*" OR "partial*
observ*" OR uncertain* OR ambigu* ) AND ( "2015" OR "2016" OR "2017" OR "2018" OR "2019" OR
"2020" OR "2021" OR "2022" OR "2023" OR "2024" OR "2025" OR "2026" )
```

E.2 IEEE Xplore (open access only)

```
( "reinforcement learning" OR "deep Q-network*" OR "Q-learning" OR "policy gradient" OR
"actor-critic" OR "proximal policy optimization" OR "Markov decision process*" OR "partially
observable Markov" OR POMDP OR "contextual bandit*" OR "multi-armed bandit*" ) AND (
"incident management" OR "incident response" OR "incident triage" OR "incident handling" OR
"incident resolution" OR "incident mitigation" OR AIOps OR "IT operations" OR "IT service
management" OR ITSM OR "site reliability" OR "service desk" OR "help desk" OR "trouble
ticket*" OR "ticket routing" OR "ticket assignment" OR "alert triage" OR "alert
prioritization" OR "security operations center" OR "automated remediation" OR ("root cause"
W/5 (microservices OR cloud OR "online services" OR "IT systems")) ) AND ( triage OR
escalation OR defer* OR "human-in-the-loop" OR "human in the loop" OR "human oversight" OR
investigation OR diagnosis OR "information gathering" OR "information acquisition" OR
remediation OR mitigation OR constraint* OR shield* OR "safe reinforcement learning" OR
"action masking" OR "partial observability" OR uncertainty OR ambiguity )
```

Appendix F

Research alignment with specialization, SDG, NDPIV & Vision 2040

F.1 Alignment with programme specialization

This research aligns with the Artificial Intelligence and Robotics specialization track of the Bachelor of Science in Computer Science. It applies deep reinforcement learning, a core method of both fields, together with decision-making under partial observability and safe reinforcement learning, a problem first studied for robots and control systems [14].

F.2 Alignment with Sustainable Development Goals

The research aligns with SDG 9, Industry, Innovation and Infrastructure, which calls for resilient infrastructure and innovation. Digital services are now part of that infrastructure, and the project studies how automation can help the people who restore them without acting outside their rules, so that faster incident handling does not cost reliability. It also contributes to Target 9.5, enhanced research and technological capability in developing countries, by building expertise in artificial intelligence for IT operations in Uganda.

F.3 Alignment with NDP IV

The Fourth National Development Plan (NDP IV) continues Uganda's Digital Transformation Programme, which aims to increase the use of information and communication technology for social and economic development [38]. As more services move online, the country depends on them staying available. The project examines a rule-governed way of introducing artificial intelligence into IT incident handling, in which the agent works within an organisation's rules and hands uncertain cases to a human expert, supporting the reliable delivery of digital services and building local skills in artificial intelligence for IT operations.

F.4 Alignment with the Uganda Vision 2040

Uganda Vision 2040 aims to transform Uganda into a competitive upper-middle-income country and treats information and communication technology as one of the foundations of that transformation [39]. This research contributes locally developed expertise in artificial intelligence for IT operations, built with and for a Ugandan institution.

Appendix G

Algorithms, Code of interest

This appendix gives the five main procedures of the study as pseudocode. Together they follow one experiment from start to finish: simulate an incident (Algorithms 1 and 2), check every proposed action against the rules (Algorithm 3), train the agent (Algorithm 4), run the strongest baseline (Algorithm 5), and evaluate every policy (Algorithm 6). The equations they refer to are in Chapter 3. The PPO learner is not listed, because it is run with the MaskablePPO implementation of sb3-contrib without change.

G.1 Simulating an Incident

Algorithm 1 Simulating one incident

Input: ambiguity level $\ell = (p_{\text{miss}}, p_{\text{noise}}, \omega)$
Output: the first observation of a new incident

procedure RESET(ℓ)
    Blend each cause's check and symptom likelihoods towards its category average by ω ▷ Eq. (3.12)
    Pick the hidden cause c from the prior, and the context x
    for each check j **do**
        if a random number is below p_{miss} **then**
            Mark check j as unavailable
        else
            Draw its result (positive or negative) for cause c
            With probability p_{noise}, reverse the result
        end if
    end for
    for each symptom e **do**
        Mark symptom e present with probability φ'_{em} for cause c ▷ Eq. (3.3), blended by ω
    end for
    Mark every check as not requested
    return the first observation: symptoms, context, and all checks not requested
end procedure

The symptoms and the results of the checks are drawn once, at the start, so asking for the same check twice always gives the same answer. Algorithm 2 then carries out each permitted action.

Algorithm 2 Taking one step in an incident

Input: a permitted action a
Output: the next observation, the reward, and whether the incident has ended

procedure STEP(a)
  if a requests check j **then**
    Reveal the result of check j; reward $= -k_j$
  else if a applies a fix **then**
    reward $= +r_{\text{correct}}$ if the fix matches c, otherwise $-\lambda_{\text{wrong}}$; end the incident
  else if a escalates **then**
    reward $= -\lambda_{\text{esc}}$; end the incident
  end if
  if the step limit H is reached **then**
    Treat the incident as escalated and end it
  end if
  return next observation, reward, ended
end procedure

G.2 Checking Actions against the Rules

Algorithm 3 Rule-based safety mechanism (review then escalate)

Input: the agent's policy π, the current observation, the ten operational rules
Output: the action carried out, which is always permitted

procedure SAFESTEP(π, observation)
  $a_1 \leftarrow$ the agent's proposal
  if no rule forbids a_1 **then**
    return STEP(a_1)
  end if
  Block a_1 and log it; reward $= -\lambda_{\text{block}}$; one step used ▷ first proposal refused
  Tell the agent it was blocked, and remove a_1 from its choices
  $a_2 \leftarrow$ the agent's second proposal
  if no rule forbids a_2 **then**
    return STEP(a_2)
  else
    Record a forced escalation
    return STEP(escalate) ▷ escalation is never forbidden
  end if
end procedure

A blocked proposal uses up one step and earns the block penalty $-\lambda_{\text{block}}$ of Equation (3.6). All the learning agents and baselines act through this procedure, except the unconstrained and penalty-only agents, which call STEP directly.

G.3 Training the Agent

Algorithm 4 Training the deep Q-network

Input: number of training steps, exploration rate ϵ, mini-batch size b, target update interval C

Output: a trained Q-network

Create the Q-network with random weights, and a copy of it as the target network
Create an empty replay buffer
repeat
    Start a new incident with RESET(ℓ) ▷ ℓ fixed, or random in the second regime
    while the incident has not ended **do**
        With probability ϵ pick a random allowed action;
            otherwise pick the allowed action with the highest Q-value
        Carry it out through SAFESTEP and store what happened in the buffer
        Take a random mini-batch of b stored steps
        for each step in the mini-batch **do**
            target $= r$ if the incident ended, otherwise $r + \gamma \max_{a'} Q_{\text{target}}(\text{next}, a')$
        end for
        Adjust the Q-network to bring its values closer to the targets ▷ Eq. (3.23)
        Every C steps, copy the Q-network into the target network
        Lower ϵ a little
    end while
until the number of training steps is reached
return the Q-network

"Allowed" always means the masked set of Equation (3.15): checks already requested and blocked proposals are excluded, both when an action is chosen and when the target is computed. Each agent is trained in this way with ten random seeds.

G.4 Confidence-Threshold Baseline

Algorithm 5 Confidence-threshold baseline

Input: the simulator's likelihoods, the prior, a confidence threshold ξ

Output: the next action

procedure DECIDE(observation)
    Update the belief over causes from the results seen so far ▷ Eq. (3.8)
    if the most likely cause has probability at least ξ **then**
        return apply the fix for that cause
    else if some checks have not been requested **then**
        return request the check expected to reduce uncertainty the most
    else
        return escalate
    end if
end procedure

This baseline is given the model that the learning agent must learn from experience, so it is the reference the agent is expected to approach. "Reduce uncertainty the most" means the largest

expected information gain, the expected fall in the entropy of the belief after the check's result is seen.

G.5 Evaluating the Policies

Algorithm 6 Evaluation protocol

Input: all policies (agent, ablations and baselines), the twelve ambiguity settings, $S = 10$, $N = 1{,}000$

Output: each measure as a mean with a 95% confidence interval

for each ambiguity setting **do**
    Generate S evaluation sets of N incidents, from seeds never used in training
end for
for each policy and each ambiguity setting **do**
    for $s = 1, \ldots, S$ **do**
        Run the policy (trained with seed s, if it learns) on evaluation set s, through SafeStep
        Record each outcome, the checks used, escalations, blocks and violations
        Compute every measure of Section 3.9 for this set
    end for
    Average each measure over the S values
    Resample the S values with replacement $B = 10{,}000$ times to get a 95% bootstrap interval
end for
return the table of means and intervals

Every policy meets the same incidents, so differences between policies are not caused by differences between incidents.

G.6 Code Repositories

- Stable-Baselines3 (DQN and PPO): `https://github.com/DLR-RM/stable-baselines3`
- sb3-contrib (MaskablePPO): `https://github.com/Stable-Baselines-Team/stable-baselines3-contrib`
- Gymnasium (environment interface): `https://github.com/Farama-Foundation/Gymnasium`

Appendix H

Other useful figures

Figure H.1 shows the flow of records through the systematic literature review that informed Chapter 2.

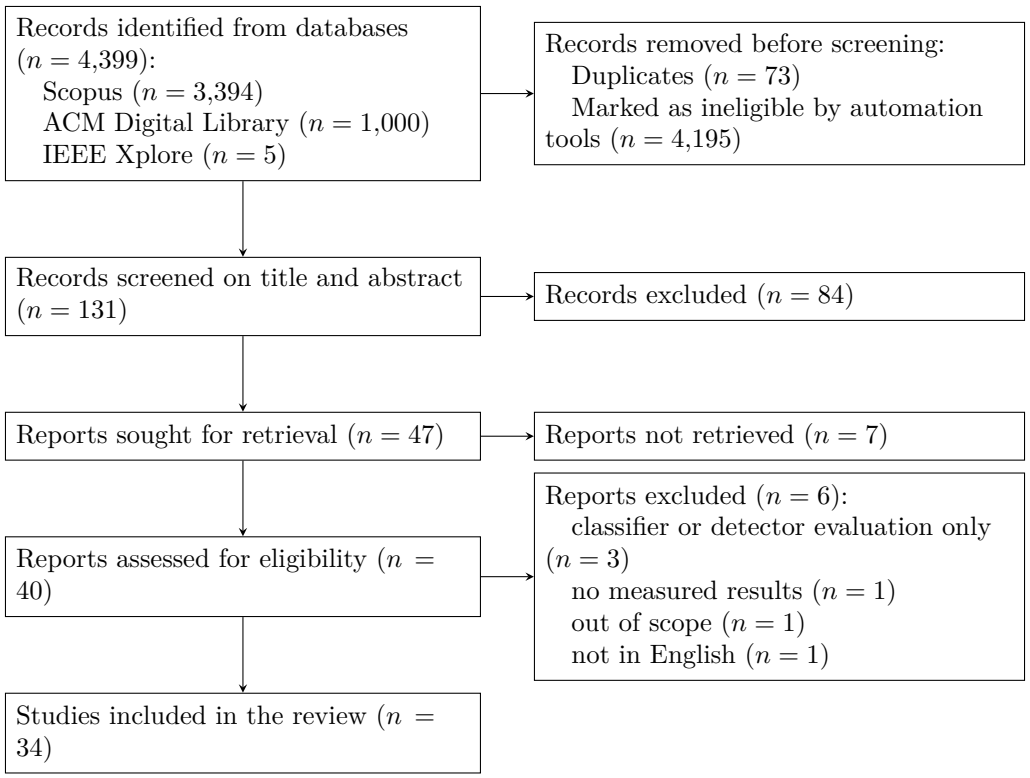


Figure H.1: PRISMA 2020 flow diagram of the systematic literature review. Citation searching was not carried out.

Records marked as ineligible by automation tools are reported as a single count, as the PRISMA 2020 flow diagram allows. IEEE Xplore returned 5 records because its search was limited to open-access records (Appendix E).

Appendix I

Research poster layout

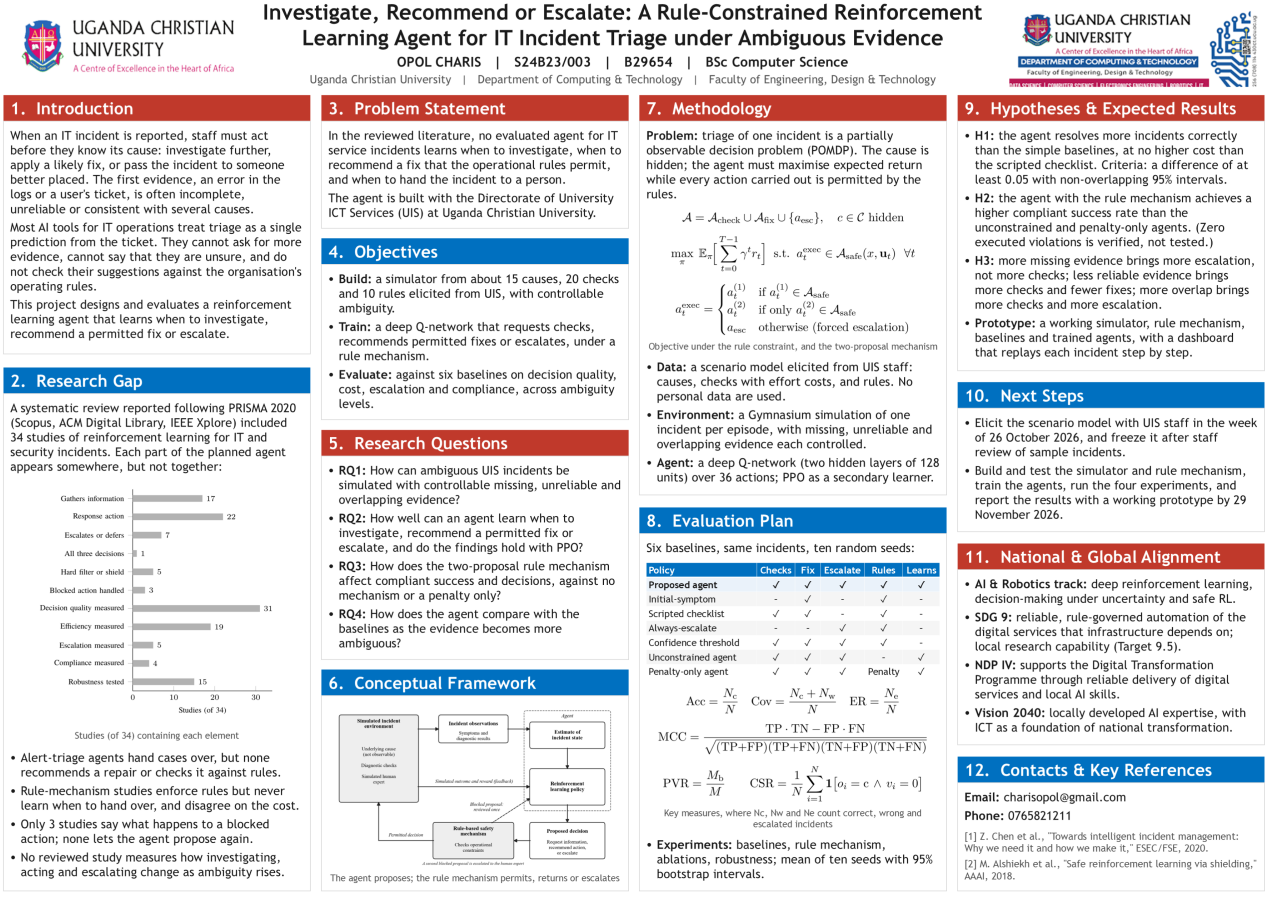


Figure I.1: Research proposal presentation poster